Lecture 03 – Model Selection

Stat 5376: Statistical Machine Learning

Ruiqi Liu

Department of Mathematics and Statistics
Texas Tech University

http://www.math.ttu.edu/~ruiqliu/

Disclaimer: These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.

Multiple Linear Regression

- More than 1 independent variable; called multiple linear regression.

$$Y = \beta_0 + \beta_1 x_{(1)} + \beta_2 x_{(2)} + \cdots + \beta_p x_{(p)} + \varepsilon$$

  Notation: I deliberately use $x_{(j)}$ (subscript with parenthesis) to denote the jth variable, instead of the jth observation.

- Observations: $(y_i, x_{i1}, x_{i2}, \ldots, x_{ip})$, $i = 1, \ldots, n$.

$$\begin{aligned}
y_1 \leftarrow Y_1 &= \beta_0 + \beta_1 x_{11} + \beta_2 x_{12} + \cdots + \beta_p x_{1p} + \varepsilon_1 \\
y_2 \leftarrow Y_2 &= \beta_0 + \beta_1 x_{21} + \beta_2 x_{22} + \cdots + \beta_p x_{2p} + \varepsilon_2 \\
&\vdots \\
y_i \leftarrow Y_i &= \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i \\
&\vdots \\
y_n \leftarrow Y_n &= \beta_0 + \beta_1 x_{n1} + \beta_2 x_{n2} + \cdots + \beta_p x_{np} + \varepsilon_n
\end{aligned}$$

- We have $(p+1)$ coefficients (1 for the intercept term and p for the independent variables), and n observations. Each observation is represented by y_i and $\boldsymbol{x}_i := (1, x_{i1}, x_{i2}, \ldots, x_{ip})^T$

Matrix Representation

- Use matrix notation to ease the derivation:

$$\boldsymbol{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{X} = \underbrace{\begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}}_{p+1 \text{ columns}} = \begin{bmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_n^T \end{bmatrix}$$

$$\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^T, \quad \boldsymbol{\varepsilon} = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n)^T$$

Rewrite model as

$$\boldsymbol{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

- The least square estimator is obtained by minimizing the Residual Sum of Squares (RSS)

$$
\begin{aligned}
RSS(\boldsymbol{\beta}) &= \sum_{i=1}^{n} \{y_i - (\widehat{\beta}_0 + \widehat{\beta}_1 x_{i1} + \widehat{\beta}_2 x_{i2} + \cdots + \widehat{\beta}_p x_{ip})\}^2 \\
&= \sum_{i=1}^{n} \{y_i - \boldsymbol{x}_i^T \boldsymbol{\beta}\}^2 \\
&= [\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}]^T [\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}]
\end{aligned}
$$

- The solution $\widehat{\boldsymbol{\beta}}$ minimizes $RSS(\boldsymbol{\beta})$, we have

$$
\mathbf{0} = \nabla RSS(\widehat{\boldsymbol{\beta}}) = -2\mathbf{X}^T[\boldsymbol{Y} - \mathbf{X}\widehat{\boldsymbol{\beta}}] = -2\mathbf{X}^T\boldsymbol{Y} + 2\mathbf{X}^T\mathbf{X}\widehat{\boldsymbol{\beta}}
$$

$$
\Rightarrow \mathbf{X}^T\mathbf{X}\widehat{\boldsymbol{\beta}} = \mathbf{X}^T\boldsymbol{Y} \Rightarrow \widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{Y}
$$

- The fitted value of y_i is $\widehat{y}_i = (1, x_{i1}, \ldots, x_{ip})\widehat{\boldsymbol{\beta}}$
- In matrix form $\widehat{\boldsymbol{Y}} = \mathbf{X}\widehat{\boldsymbol{\beta}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{Y} = H\boldsymbol{Y}$
- Here $H = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ is called hat matrix.

ANOVA Decomposition

- Define notations:

$$TSS = \sum_{i=1}^{n} (y_i - \overline{y})^2, \qquad \text{Total sum of squares}$$

$$RSS = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \qquad \text{Residual sum of squares}$$

$$SS_{reg} = \sum_{i=1}^{n} (\hat{y}_i - \overline{y})^2, \qquad \text{Sum of squares in regression}$$

- ANOVA Decomposition:

$$\underbrace{TSS}_{\text{Total variations in } Y} = \underbrace{SS_{reg}}_{\text{Variations in } Y \text{ explained by model}} + \underbrace{RSS}_{\text{Variations in } Y \text{ unexplained by model}}$$

Definition of R^2

The R^2 is defined as

$$R^2 = \frac{SS_{reg}}{TSS} = 1 - \frac{RSS}{TSS}$$

- R^2 is between 0 and 1;
- The bigger the R^2, the better (loosely speaking..., we will see an issue of R^2 later);

Some Comments on R^2

- R^2 can be used to measure the strength of association;
- R^2 can only measure LINEAR relationship;
- If the relationship between x and y is not linear, R^2 is useless!
- R^2 and the scatter plot need to be used together! (The next four plots have exactly the same R^2!)

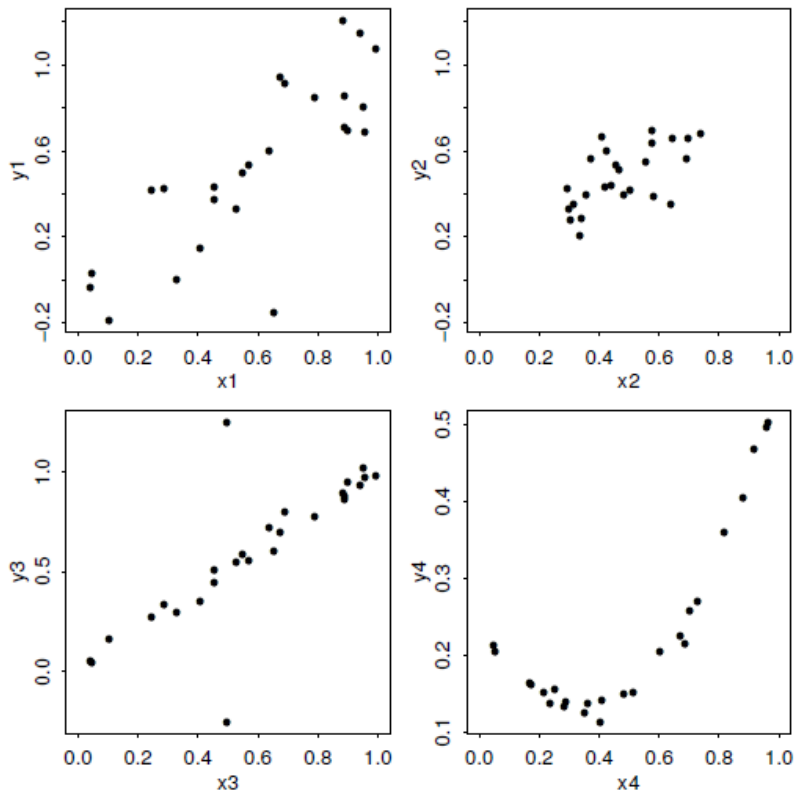


Figure 2.3 Four simulated datasets where R^2 is about 0.65. The plot on the upper

Major Problem with R^2

Recall the definition of R^2

$$R^2 = 1 - \frac{RSS}{TSS}$$

$$RSS(\beta_0, \ldots, \beta_p) = \min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \left(y_i - \beta_0 - x_{1i}\beta_1 - \cdots - x_{pi}\beta_p\right)^2.$$

$$TSS = \sum_{i=1}^{n} (y_i - \overline{y})^2.$$

Suppose we add an addition predictor x_{p+1} into our regression model (which can have nothing to do with the response variable Y at all), then the new fitted RSS would be

$$RSS(\beta_0, \ldots, \beta_p, \beta_{p+1}) = \min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \left(y_i - \beta_0 - x_{1i}\beta_1 - \cdots - x_{pi}\beta_p - x_{p+1,i}\beta_{p+1}\right)^2.$$

- $RSS(\beta_0, \ldots, \beta_p, \beta_{p+1}) \leq RSS(\beta_0, \ldots, \beta_p)$ is always true;
- TSS does not change when adding a new predictor;
- R^2 will always increase by adding a new predictor (no matter what predictor!);

Model Over-Fitting

- To show the problem, let's see a simple example;
- Let's generate $n = 6$ data points from the model

$$y_i = 1 + 2x_i + \varepsilon_i, \quad \varepsilon_i \sim N(0, 1^2), \quad i = 1, \ldots, 6.$$

- Lets first fit a regression model (1)

$$y = \beta_0 + \beta_1 x$$

- Lets first fit a regression model (2)

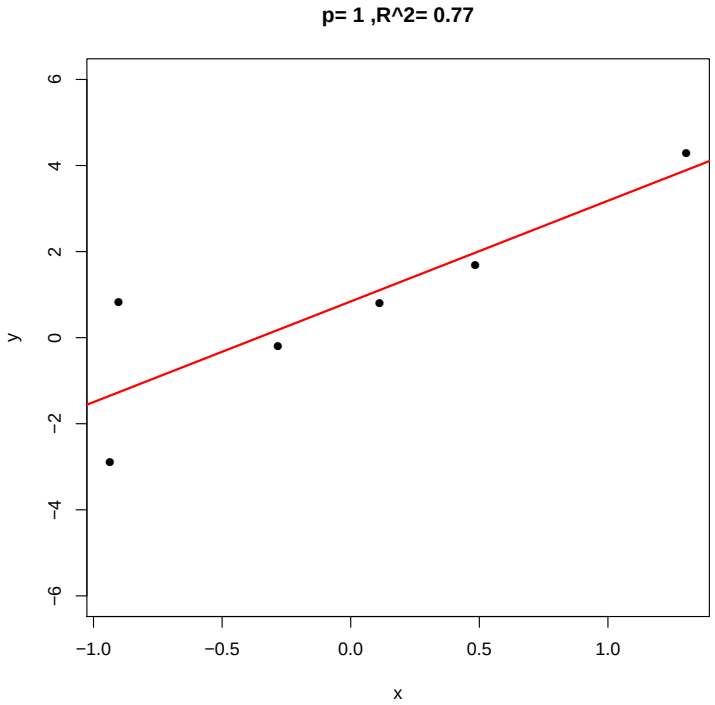
$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

- $\vdots$
- Lets first fit a regression model (5)

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_5 x^5$$

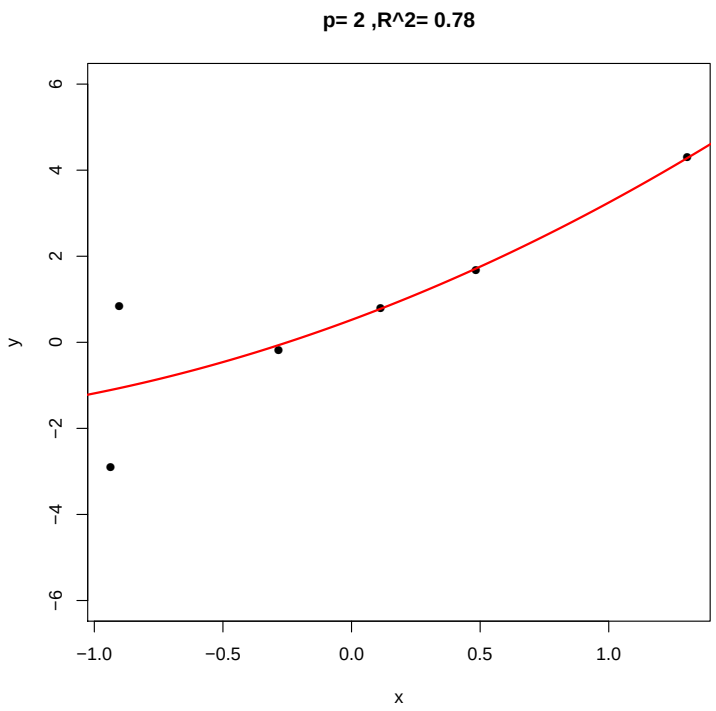
- Add a new predictor x^p one at a time, $p = 1, \ldots, 5$.

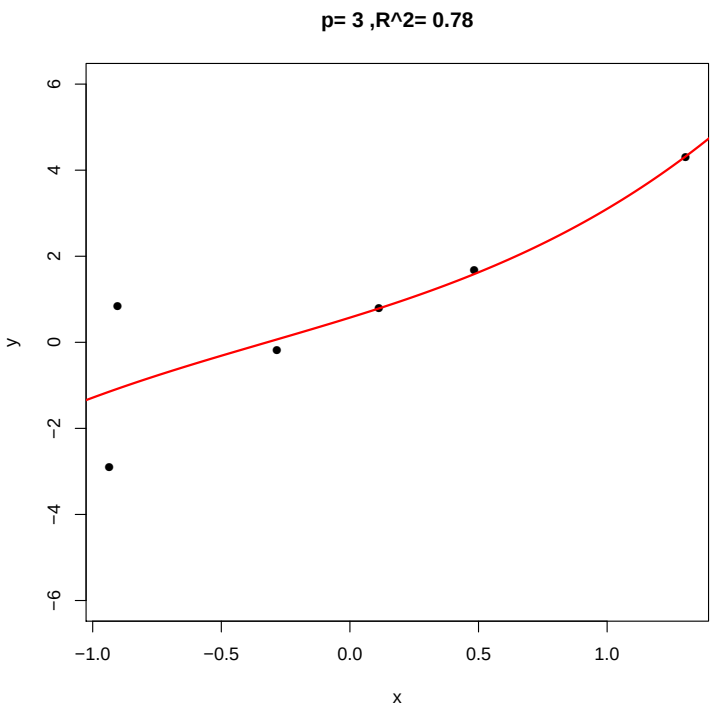
Model Over-Fitting



- $p = 1$ fits the model reasonably well;

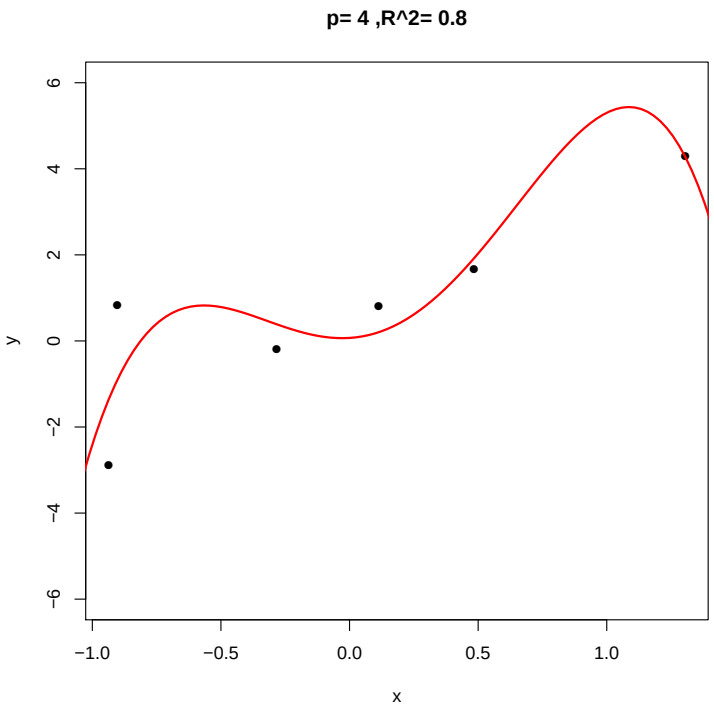
Model Over-Fitting

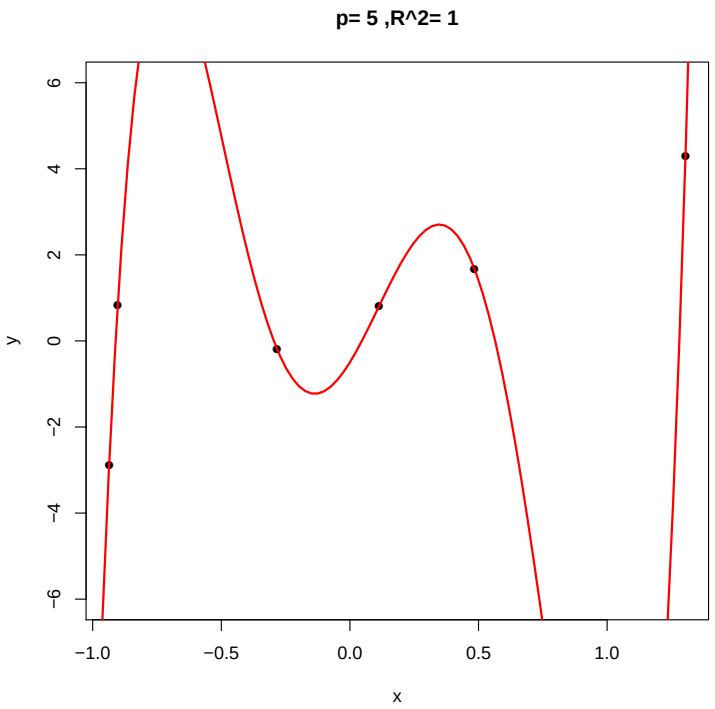




- $p = 2, 3$ R^2 increase a little bit;
- The model fit looks ok!

Model Over-Fitting





- $p = 6$: $R^2 = 1$;
- But the model fit is horrible!
- This is the case of overfitting!

Model Over-Fitting

- Over-fitting: usually happens when your sample size n is small but the number of parameter p is large!
- For the regression model $\boldsymbol{Y} = \mathbf{X}\boldsymbol{\beta} + \varepsilon$, the over-fitting happens because
  - when your model is complicated enough (i.e. p large enough), you can fit the error term ε very well!
  - This is not what you want! You want to fit the signal $\mathbf{X}\boldsymbol{\beta}$ really well, not ε;
  - Over-fitting means your model can fit the “training data" really well, but when you try to use your model to do prediction on another data set, the performance could be horrible!
- How to avoid over-fitting?
  - Use some model selection criterion: AIC, BIC, (idea: to penalize the model complexity!)
  - Use cross validation: use part of your data as “training data" and evaluate the predictive performance of the fitted model using “validation data" that has not been used.

Adjusted R^2

- Define the adjusted R^2 AS

$$R^2_{adj} = 1 - \frac{RSS/(n-p-1)}{TSS/(n-1)} = 1 - \frac{MSE}{MST},$$

- R^2_{adj} For a “bigger" model (with a larger p) will be penalized;
- R^2_{adj}: maybe less then 0; (when would that happen?)

Information Criterion

The idea of R^2_{adj} gives us a good idea. To evaluate a model is good or not, we need to

- have a small value of RSS: this mean your model fit your data well;
- penalize the model complexity: to make sure your model is not too big;
- Based on this idea, we have following criterion:
  - Akaike Information Criterion (AIC):

$$AIC = \frac{1}{n}(RSS/\hat{\sigma}^2 + 2 \times d),$$

  - Bayesian Information Criterion (BIC):

$$BIC = \frac{1}{n}(RSS/\hat{\sigma}^2 + \log n \times d),$$

  - Mallow's CP (C_p):

$$C_p = \frac{1}{n}(RSS + 2 \times d \times \hat{\sigma}^2),$$

    - $\hat{\sigma}^2$: the sample variance estimator under the full model:

$$RSS_{Full}/(n - p_{Full} - 1)$$

    - d: the number of parameters in the model (for linear model, is $p + 1$).

Subset Selection

- Now that we have some criterions to evaluate a “good" model, we can choose the best model as the one with the optimal score (AIC,BIC or C_p);
- If we have p potential predictors, then total number of possible models 2^p;
- Subset selection: choose the best one among 2^p candidate models;
- This is usually impossible to do for even moderate p; (e.g: $2^{10} = 1024$!)

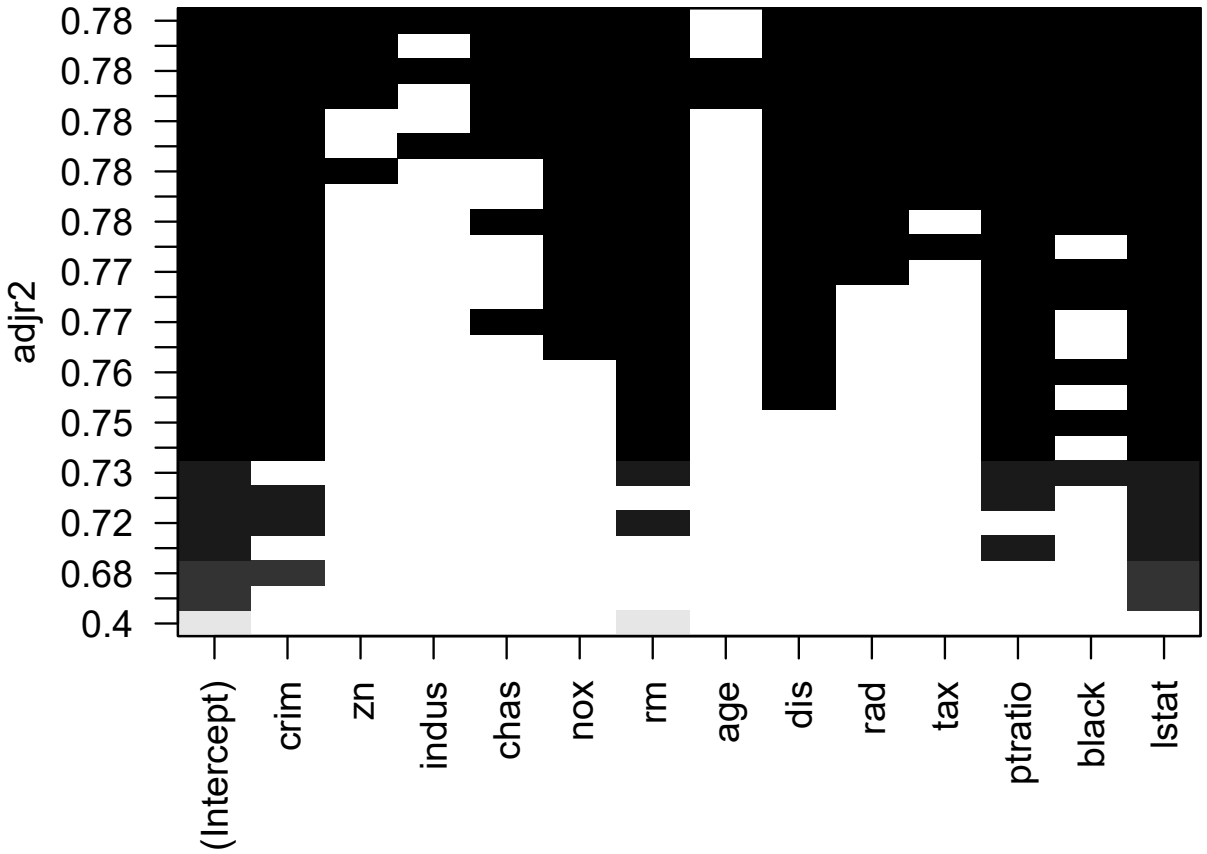
Boston Housing Data

- You need to install R-packages “leaps" to perform subset selection;
- The function *regsubsets()* in the package finds the best subset of predictors for every size.

```
rm(list=ls())
library(MASS)
library(leaps)
data(Boston)
## Optional arguments:
##nbest=2 will find the two best for each size
##nvmax=13 will find them for all sizes up to 13
##force.in=c(6,13)will force variables 6 and 13 in the model(RM, LSTAT)
## Example:
summary(regsubsets(log(medv) ~., data=Boston),nbset=1)
summary(regsubsets(log(medv) ~., data=Boston,
                   nbest=3, nvmax=13, force.in=c(6,13)))

## A graphical way of looking at these models:
plot(regsubsets(log(medv) ~., data=Boston,
                nbest=2, nvmax=13), scale="adjr2")
```

Boston Housing Data

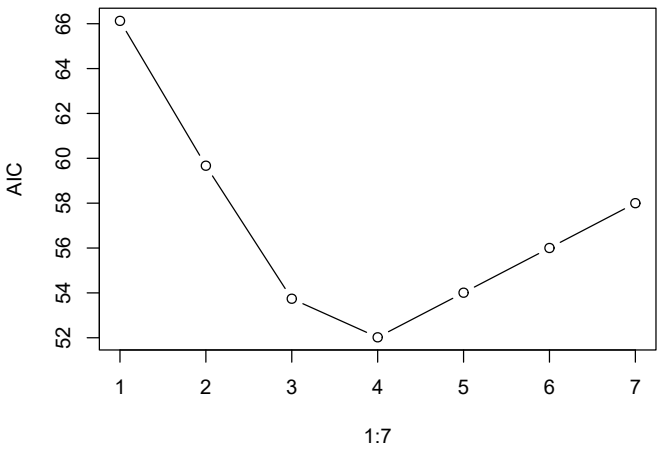
- There are in total $2^{13} = 8192$ models;
- For a model of each size, two best models are presented ($13 \times 2 = 26$);

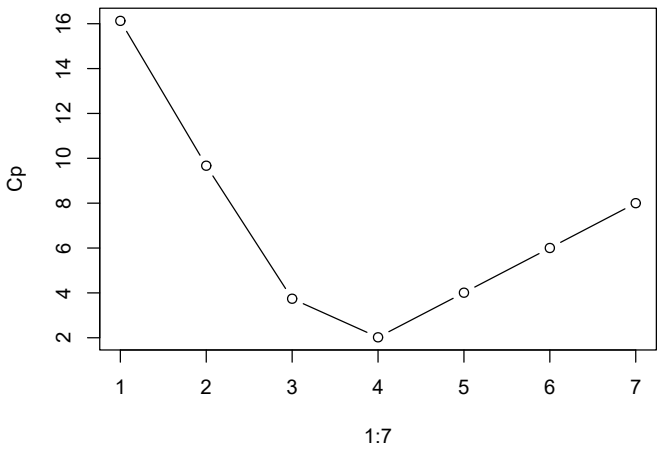


State Data

```
rm(list=ls())
library(leaps)
statedata <- data.frame(state.x77,row.names=state.abb)
b <- regsubsets(Life.Exp~.,data=statedata)
rs <- summary(b)
rs$which
  (Intercept) Population Income Illiteracy Murder HS.Grad Frost  Area
1        TRUE      FALSE  FALSE      FALSE   TRUE   FALSE FALSE FALSE
2        TRUE      FALSE  FALSE      FALSE   TRUE    TRUE FALSE FALSE
3        TRUE      FALSE  FALSE      FALSE   TRUE    TRUE  TRUE FALSE
4        TRUE       TRUE  FALSE      FALSE   TRUE    TRUE  TRUE FALSE
5        TRUE       TRUE   TRUE      FALSE   TRUE    TRUE  TRUE FALSE
6        TRUE       TRUE   TRUE       TRUE   TRUE    TRUE  TRUE FALSE
7        TRUE       TRUE   TRUE       TRUE   TRUE    TRUE  TRUE  TRUE
####calculating sigma_hat
lmod=lm(Life.Exp~.,data=statedata)
RSS_full=sum((lmod$residuals)^2)
sigma_hat2=RSS_full/lmod$df.residual
####calculating AIC
AIC=rs$rss/sigma_hat2+2*(2:8)
plot(1:7, AIC,type="b",ylab="AIC")
####Plot Cp
  plot(1:7,rs$cp,ylab="Cp"type="b")
```

State Data





- Minimizing AIC and C_p give the same best model!

Forward/Backward/Stepwise Selection

- When p is large, it is impossible to compare all 2^p models;
- We need to create a “path" to search over some (not all) models;
- Forward Selection
  - we start with no predictors in the model;
  - We then add the one variable that leads to the best score (AIC,BIC,or C_p);
  - We continue adding variables one at a time this way until no predictors can be added in the model;
- Backward Selection
  - we start with full model;
  - We then drop one variable that leads to the best score (AIC,BIC,or C_p);
  - We continue dropping variables one at a time this way until no predictors can be dropped in the model;
- Stepwise Selection
  - is a combination of backward elimination and forward selection;
  - At each stage a variable may be added or removed; (that is, we can regret our earlier decisions);
  - Stepwise selection can be conducted forward or backward;

Some Comments on Stepwise Selection

- Because of the “one-at-a-time" nature of adding/dropping variables, it is possible to miss the “optimal" model;
- Froward selection and backward selection my end up with different “best" models;
- Backward selection is infeasible when $p > n$, when the full model cannot be fitted;
- Forward selection or Stepwise forward selection is preferred when p is large;

State Data

- You need to install R-packages “Rcmdr" to perform stepwise selection;
- The function *stepwise()* in the package finds the best subset of predictors for every size.

```
library(Rcmdr)
data(state)
statedata <- data.frame(state.x77,row.names=state.abb)
###Fit the full model
lmod <- lm(Life.Exp ~ ., data=statedata)
##Forward selection based on AIC
stepwise(lmod,direction="forward",criterion="AIC")
##Backward selection based on AIC
stepwise(lmod,direction="backward",criterion="AIC")
##Stepwise backward selection based on AIC
stepwise(lmod,direction="backward/forward",criterion="AIC")
##Stepwise forwad selection based on AIC
stepwise(lmod,direction="forward/backward",criterion="AIC")
```

State Data

- Best model selected by Forward selection with AIC

```
Step:  AIC=-28.16
Life.Exp ~ Murder + HS.Grad + Frost + Population
```

- Best model selected by Backward selection with AIC

```
Step:  AIC=-28.16
Life.Exp ~ Population + Murder + HS.Grad + Frost
```

- Best model selected by Stepwise Backward selection with AIC

```
Step:  AIC=-28.16
Life.Exp ~ Population + Murder + HS.Grad + Frost
```

- They all select the same best model based on AIC (not always the case);
- You can also use “BIC" as your selection criterion, you may end up with a different “best" model;

Another Way to Look at Model Selection

- Consider the multiple regression model

$$y_i = \beta_0 + x_1\beta_1 + \cdots + x_p\beta_p + \varepsilon_i;$$

- The idea of model selection is to determine which subset of predictors should be kept in the model;
- Another way to look at it: if we can estimated some β_j's as 0, then the corresponding predictors x_j's will be excluded from the model, hence finish model selection;
- However, the least square estimator $\widehat{\boldsymbol{\beta}}$ will not do that because none of its elements is zero;
- We do observed that some elements of $\widehat{\boldsymbol{\beta}}$ are very small, can we force (shrink) them to 0?

Attempt I: Ridge Regression

- Recall $\boldsymbol{\beta} = (\beta_0, \beta_1, \ldots, \beta_p)^T$
- The ridge regression estimator chooses the $\boldsymbol{\beta}$ that minimizes

$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) \quad \text{subject to } \sum_{j=1}^p \beta_j^2 \leq t$$

  for some $t > 0$, denote it as $\widehat{\boldsymbol{\beta}}_{ridge}$;
- Usually, we do not impose restriction on the intercept coefficient β_0.
- If t is large enough such that $\sum_{j=1}^p \hat{\beta}_{ols,j}^2 \leq t$, then $\widehat{\boldsymbol{\beta}}_{ridge} = \widehat{\boldsymbol{\beta}}_{ols}$;
- When $t \approx 0$, all $\beta_j \approx 0$, hence forcing $\widehat{\boldsymbol{\beta}}_{ridge}$ towards 0;

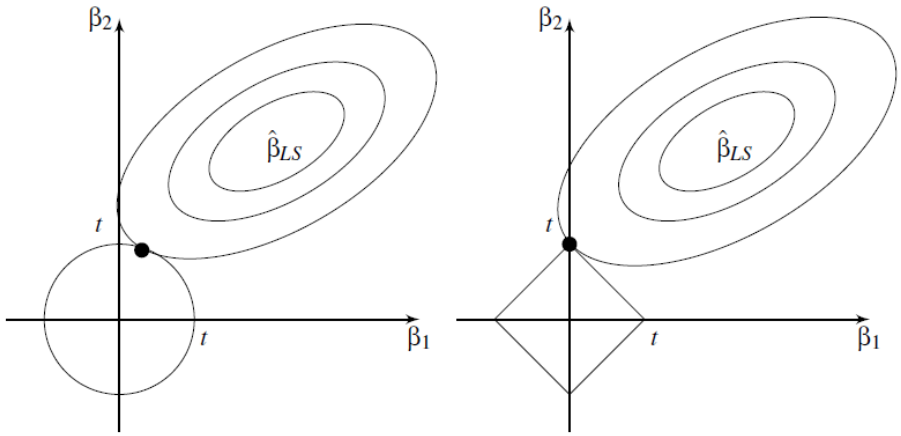


Figure 11.9 Ridge and lasso regression are illustrated. On the left, confidence ellipses of increasing level are plotted around the least squares estimate. The largest ellipse intersects the circle of radius t at the ridge estimate. On the right, the largest ellipse intersects the square at the lasso estimate.

Attempt I: Ridge Regression

- The ridge regression estimator chooses the $\boldsymbol{\beta}$ that minimizes

$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) \quad \text{subject to } \boldsymbol{\beta}^T\mathbf{R}\boldsymbol{\beta} \leq t,$$

  where

$$\mathbf{R} = \begin{pmatrix} 0 & & & \\ & 1 & & \\ & & \ddots & \\ & & & 1 \end{pmatrix} \in \mathbb{R}^{(p+1)\times(p+1)}$$

- KKT Conditions:
  - $\mathbf{X}^T(\mathbf{X}\boldsymbol{\beta} - \boldsymbol{Y}^T) + \lambda\mathbf{R}\boldsymbol{\beta} = 0$
  - $\lambda(\boldsymbol{\beta}^T\mathbf{R}\boldsymbol{\beta} - t) = 0$
  - $\boldsymbol{\beta}^T\mathbf{R}\boldsymbol{\beta} \leq t$
  - $\lambda \geq 0$
- We consider two cases for the solution $\widehat{\boldsymbol{\beta}}_t$:
  - If $\widehat{\boldsymbol{\beta}}_t^T\mathbf{R}\widehat{\boldsymbol{\beta}}_t < t$, then $\lambda = 0$. So

$$\widehat{\boldsymbol{\beta}}_t = \operatorname{argmin}_{\boldsymbol{\beta}}(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})$$

  - If $\widehat{\boldsymbol{\beta}}_t^T\mathbf{R}\widehat{\boldsymbol{\beta}}_t = t$, then $\lambda > 0$. So

$$\widehat{\boldsymbol{\beta}}_t = \operatorname{argmin}_{\boldsymbol{\beta}}(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}^T\mathbf{R}\boldsymbol{\beta}$$

Attempt I: Ridge Regression

- We can transfer the constrained problem to an unconstrained problem
$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}^T\mathbf{R}\boldsymbol{\beta},$$
where there is a $\lambda \geq 0$ corresponding to each t.
- The solution of the above unconstrained optimization problem is
$$\widehat{\boldsymbol{\beta}}_{ridge} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{R})^{-1}\mathbf{X}^T\boldsymbol{Y},$$
- When $\lambda = 0$, $\widehat{\boldsymbol{\beta}}_{ridge} = \widehat{\boldsymbol{\beta}}_{ols}$;
- When $\lambda = \infty$, $\widehat{\boldsymbol{\beta}}_{ridge} = 0$ (although this usually will not happen);
- Choice of λ is critical!

Statistical Properties of $\widehat{\boldsymbol{\beta}}_{ridge}$

- It is easy to show that

$$Bias(\widehat{\boldsymbol{\beta}}_{ridge}) = [(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{R})^{-1}\mathbf{X}^T\mathbf{X} - \mathbf{I}]\boldsymbol{\beta} \neq 0,$$

$$Var(\widehat{\boldsymbol{\beta}}_{ridge}) = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{R})^{-1}\mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{R})^{-1}$$

- When $\lambda = 0$,

$$Bias(\widehat{\boldsymbol{\beta}}_{ridge}) = 0 \text{(smallest) and } Var(\widehat{\boldsymbol{\beta}}_{ridge}) = (\mathbf{X}^T\mathbf{X})^{-1} \text{(largest)}$$

- As λ increase from 0 to ∞,

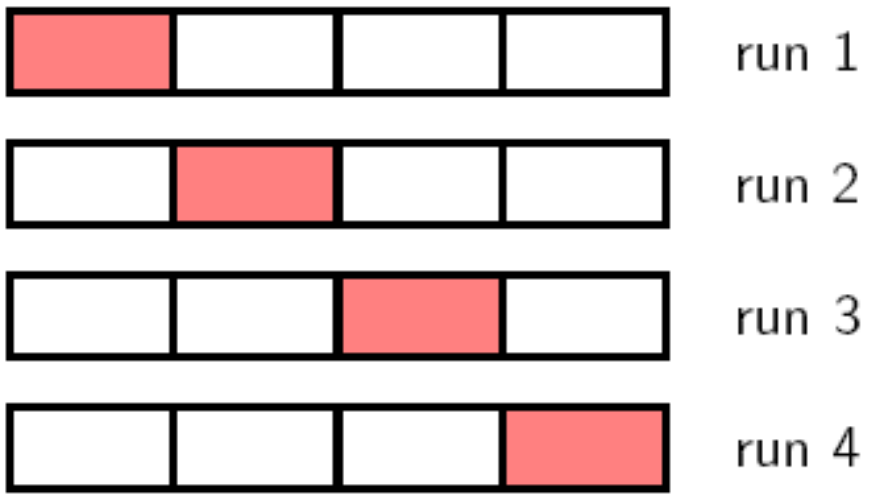
$$Bias(\widehat{\boldsymbol{\beta}}_{ridge}) \text{ increases}$$

$$Var(\widehat{\boldsymbol{\beta}}_{ridge}) \text{ decreases}$$

- λ controls bias and variance tradeoff! What should be an optimal value of λ?
- Choice of λ is critical! But how to choose?

Cross Validation

- Recall that the main motivation for studying regression is prediction;
- The residual sum of squares $RSS(\widehat{\boldsymbol{\beta}})$ only gives the “training error" not a good prediction error;
- We need to find an honest estimation of prediction accuracy
  - m-fold cross validation:



  - the data are randomly divided into m groups;
  - $m-1$ groups are used to fit the model and then to predict the left-out group;
  - The over all prediction error can be evaluated by repeating above procedure m-times;

Generalized Cross Validation

- We need to find an honest estimation of prediction accuracy
  - Generalized cross-validation score is defined as

$$GCV(\lambda) = \frac{\frac{1}{n} RSS(\widehat{\beta}_{ridge})}{\{1 - n^{-1} tr(\mathbf{H}(\lambda))\}^2}$$

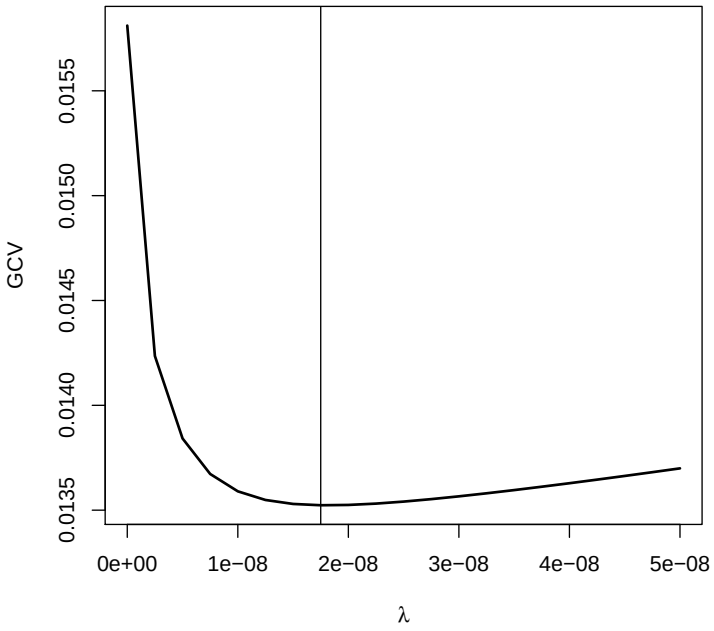
    - $RSS(\widehat{\beta}_{ridge}) = (\boldsymbol{Y} - \mathbf{X}\widehat{\boldsymbol{\beta}}_{ridge})^T(\boldsymbol{Y} - \mathbf{X}\widehat{\boldsymbol{\beta}}_{ridge})$;
    - $\mathbf{H}(\lambda) = \mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{R})^{-1}\mathbf{X}^T$: the Hat matrix for ridge regression;
    - $tr(\mathbf{H}(\lambda))$: the effective degrees of freedom;
    - n: total number of observations
- GCV is more computationally efficient than m-fold cross-validation; (why?)
- GCV is more stable than m-fold cross-validation; (why?)

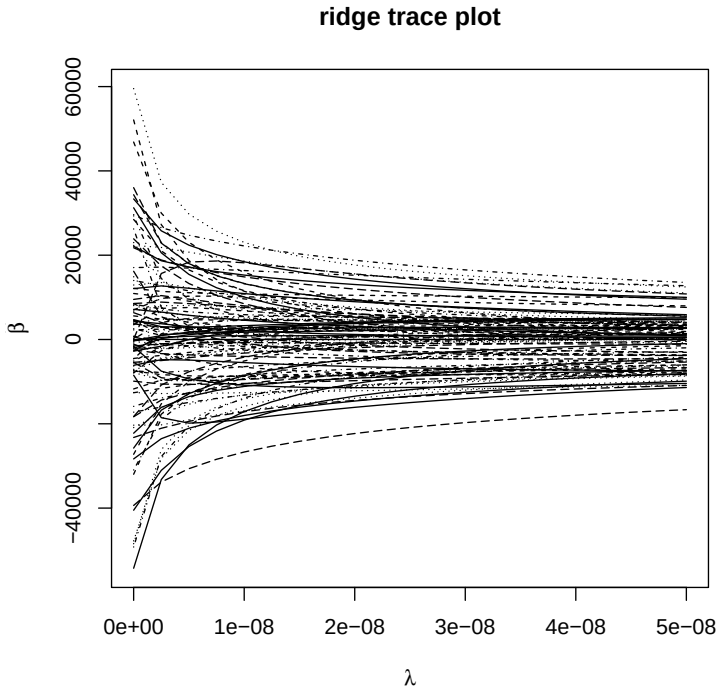
Ridge regression in R: meatspec data

- You need to install R-packages “MASS";
- The function *lm.ridge()* in the package performs ridge regression.

```
library(faraway)
library(MASS)
data(meatspec, package="faraway")
trainmeat <- meatspec[1:172,] #Use first 172 obs. as training data
testmeat <- meatspec[173:215,] #Use the rest obs. as test data
lambda <- seq(0, 5e-8, len=21)##Candidate values for lambda
#Ridge regression
rgmod <- lm.ridge(fat ~ ., trainmeat, lambda =lambda )
which.min(rgmod$GCV)
> 8
lambda[8]
> 1.75e-08
plot(lambda,rgmod$GCV,type="l",lwd=2,ylab="GCV",
xlab=expression(lambda))
abline(v=1.75e-08)
matplot(rgmod$lambda, coef(rgmod), type="l",
xlab=expression(lambda),ylab=expression(hat(beta)),col=1)
abline(v=1.75e-08)
```

Meatspec Data





- Some experimentation was necessary to determine the appropriate range of λ
- The potential range for λ is $[0, 5 \times 10^{-8}]$;
- The $\lambda = 1.75 \times 10^{-08}$ gives the best GCV value;
- The best GCV value is 0.01352349;

Meatspec Data

- To evaluate the performance, we first calculate the training error for $\lambda = 1.75 \times 10^{-08}$;

```
ypred <- cbind(1,as.matrix(trainmeat[,-101]))%*%coef(rgmod)[8,]
> sqrt(mean((ypred-trainmeat$fat)^2))
[1] 0.8024395
```

- The prediction error of the test data with $\lambda = 1.75 \times 10^{-08}$

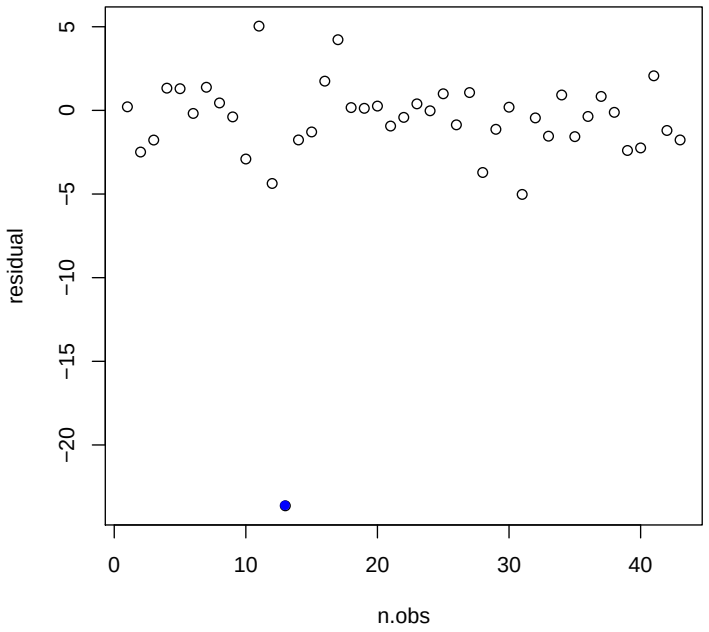
```
ypred <- cbind(1,as.matrix(testmeat[,-101])) %*% coef(rgmod)[8,]
>  sqrt(mean((ypred-testmeat$fat)^2))
[1] 4.101066
```

- We note here the prediction error (4.101066) is a lot worse than the training error (0.8024395);
- Although this is sort of expected, it still seems unusual;

Meatspec Data

- To find out what might be wrong, we plot the residual of the test data:

```
res <- ypred-testmeat$fat
plot(res,ylab="residual",xlab="n.obs")
points(13,res[13],pch=16,col="blue")
```



- It seems that observation # 13 is the only bad fit;
- After removing 13th point, the predition error on test data becomes

```
>  sqrt(mean((ypred[-13]-testmeat$fat[-13])^2))
[1]  1.979462
```

Attempt II: Lasso

- Although, the ridge regression can somehow shrink $\widehat{\boldsymbol{\beta}}_{ridge}$ towards 0, none of elements in $\widehat{\boldsymbol{\beta}}_{ridge}$ can be exactly 0;
- In a sense, ridge regression cannot be used for the purpose of model selection since it does not eliminate any predictors;
- To overcome this, Tibshirani (1996) introduced “Lasso": the least absolute shrinkage and selection operator;
- The Lasso estimator is obtained by minimizing

$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) \quad \text{subject to } \sum_{j=1}^{p} |\beta_j| \leq t$$

  for some $t > 0$, denote it as $\widehat{\boldsymbol{\beta}}_{lasso}$;
- If t is large enough such that $\sum_{j=1}^{p} \hat{\beta}_{ols,j}^2 \leq t$, then $\widehat{\boldsymbol{\beta}}_{lasso} = \widehat{\boldsymbol{\beta}}_{ols}$;
- When $t \approx 0$, all $\beta_j \approx 0$, hence forcing $\widehat{\boldsymbol{\beta}}_{lasso}$ towards 0;

Attempt II: Lasso

- Unlike the ridge regression estimator $\widehat{\boldsymbol{\beta}}_{ridge}$, elements in $\widehat{\boldsymbol{\beta}}_{lasso}$ can be exactly 0, hence fulfill the purpose of “model selection";
- As t increases, more variables are added to the model and their coefficients become larger.

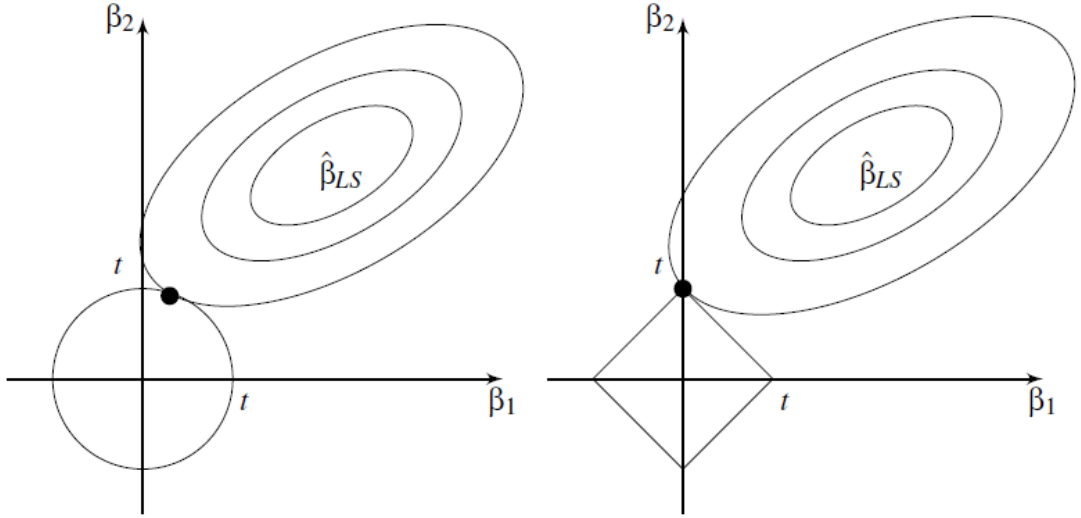


Figure 11.9 Ridge and lasso regression are illustrated. On the left, confidence ellipses of increasing level are plotted around the least squares estimate. The largest ellipse intersects the circle of radius t at the ridge estimate. On the right, the largest ellipse intersects the square at the lasso estimate.

Attempt II: Lasso

- Fun facts: The name, lasso, is an acronym. It refers to the rope used by cowboys to pull cattle from the herd just as the statistician pull estimated coefficients toward 0.

Attempt II: Lasso

- Using a Lagrange Multiplier, minimizing the constrained problem is equivalent to minimizing
$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=1}^{p} |\beta_j|$$
for some $\lambda \geq 0$;
- Choosing t is equivalent to choosing λ.
- No closed form solution exist unless $\mathbf{X}^T\mathbf{X} = \mathbf{I}$;
- The use of lasso is most appropriate when we believe the effects are sparse: that the response can be explained by a small number of predictors with the rest having no effect.
- The choice of λ or t is important; we can use Cross-validation, Generalized cross-validation.

Attempt II: Lasso

- A view on lasso when $\mathbf{X}^T\mathbf{X} = \mathbf{I}$, the orthonormal design.
- Exclude intercept β_0.
- Recall $\widehat{\boldsymbol{\beta}}_{ols} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{Y} = \mathbf{X}^T\boldsymbol{Y}$
- Taking subgradient of $\frac{1}{2}(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda\sum_{j=1}^n |\beta_j|$, we have
$$0 \in -\mathbf{X}^T(Y - \mathbf{X}\boldsymbol{\beta}) + \lambda\mathbf{s},$$
where
$$\mathbf{s} = \begin{pmatrix} s_1 \\ \vdots \\ s_p \end{pmatrix} \text{ with } s_j = \begin{cases} 1 & \text{if } \beta_j > 0 \\ -1 & \text{if } \beta_j < 0 \\ [-1, 1] & \text{if } \beta_j = 0 \end{cases}$$
- Therefore, the solution should satisfy $0 \in -\widehat{\boldsymbol{\beta}}_{ols} + \boldsymbol{\beta} + \lambda\mathbf{s}$.
- In scalar form: $0 \in -\widehat{\beta}_{ols,j} + \beta_j + \lambda s_j$.
- So the solution is $\widehat{\beta}_{lasso,j} = \begin{cases} \widehat{\beta}_{ols,j} - \lambda & \text{if } \widehat{\beta}_{lasso,j} > 0 \Leftrightarrow \widehat{\beta}_{ols,j} > \lambda \\ \widehat{\beta}_{ols,j} + \lambda & \text{if } \widehat{\beta}_{lasso,j} < 0 \Leftrightarrow \widehat{\beta}_{ols,j} < -\lambda \\ 0 & \text{if } \widehat{\beta}_{lasso,j} = 0 \Leftrightarrow |\widehat{\beta}_{ols,j}| \leq \lambda \end{cases}$
- λ is a threshold for shrinkage.

QP Algorithm for Lasso

- Lasso can be viewed as a Quadratic Programming problem. Recall

$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) \quad \text{subject to } \sum_{j=1}^p |\beta_j| \le t$$

- In scalar form:

$$\sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij}\beta_j)^2 \quad \text{subject to } \sum_{j=1}^p |\beta_j| \le t$$

- Define $\beta_j = \beta_j^+ - \beta_j^-$ where $\beta_j^+ = \max(0, \beta_j)$ and $\beta_j^- = \max(0, -\beta_j)$
- So $|\beta_j| = \beta_j^+ + \beta_j^-$.
- The problem becomes

$$\begin{aligned} & \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij}\beta_j^+ + \sum_{j=1}^p x_{ij}\beta_j^-)^2 \\ \text{subject to} \quad & \sum_{j=1}^p (\beta_j^+ + \beta_j^-) \le t \\ & \beta_j^+, \beta_j^- \ge 0, \text{ for } j = 1, \ldots, p. \end{aligned}$$

- This is a standard QP, which can be solved by active-set algorithm.

QP Algorithm for Lasso

- Issue with QP algorithm
  - When p is large ($p > n$), solving the QP problem is inefficient.
  - We need to try different values of t (or equivalently λ).
- Need a method to calculate $\widehat{\boldsymbol{\beta}}_{lasso}$ corresponding to a full span of λ values.

LARS Algorithm

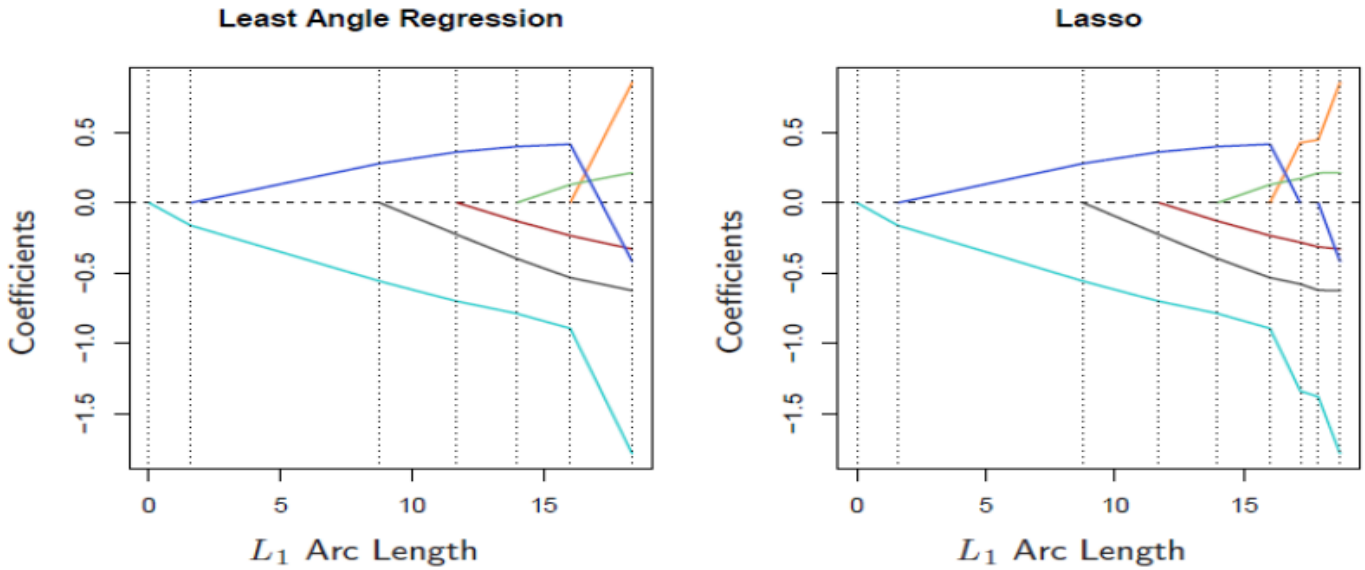
- Least Angle Regression (Lars): $y = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p + \epsilon$.
- Step 1: Let $r = y$ and initialize $\widehat{\beta}_j = 0$ for $j = 0, 1, \ldots, p$.
- Step 2: Find the variable x_{j_1} most correlated with r.
  - find the variable x_{j_1} with the the largest $|corr(x_j, r)|$ among $j = 1, \ldots, p$. Define the active set $\mathcal{A} = \{j_1\}$
- Step 3: Perform a linear regression $r \sim x_{j_1}$ (with intercept), and let (b_0, b_{j_1}) be the corresponding estimated coefficients.
- Step 4: Slowly increasing α from 0 and update $(\widehat{\beta}_0, \widehat{\beta}_{j_1}) = \alpha(b_0, b_{j_1})$, $r = y - \widehat{\beta}_0 - \widehat{\beta}_{j_1} x_{j_1}$ until there is another variable x_{j_2} with $|corr(x_{j_1}, r)| = |corr(x_{j_1}, r)|$.
  - When increasing α_1, $corr(x_{j_1}, r)$ will decrease.

LARS Algorithm

- Step 5: Update the active set $\mathcal{A} = \{j_1, j_2\}$. Perform a linear regression $r \sim x_{j_1} + x_{j_2}$ (with intercept), and let (b_0, b_{j_1}, b_{j_2}) be the estimated coefficients.
- Step 6: Slowly increasing α from 0 and update $(\widehat{\beta}_0, \widehat{\beta}_{j_1}, \widehat{\beta}_{j_2}) = \alpha(b_0, b_{j_1}, b_{j_2})$, $r = y - \widehat{\beta}_0 - \widehat{\beta}_{j_1} x_{j_1} - \widehat{\beta}_{j_2} x_{j_2}$ until there is another variable x_{j_3} with $|corr(x_{j_3}, r)| = |corr(x_{j_1}, r)| = |corr(x_{j_1}, r)|$.
  - It can be verify that when increasing α, the correlations $corr(x_{j_1}, r)$, $corr(x_{j_2}, r)$ are all equal for all variables in the active set.
  - Increasing α will decrease the correlations $corr(x_{j_1}, r)$, $corr(x_{j_2}, r)$.
- Repeat Step 1-6 until all the variables are in the active set.
  - If $p + 1 \leq n$, the final model will contains p variables plus one intercept.
  - If $p + 1 > n$, the final model will contains $n - 1$ variables plus one intercept.

Modified LARS Algorithm for Lasso

- But the above procedure only gives the Lars solution, not Lasso.
- Lasso Modification: During the procedure, if $\widehat{\beta}_j$ hits zero, we remove j from the active set $\mathcal{A}$.
- Difference of the **Solution Path** between Lars and Lasso modification algorithms:



- Recall, we do not have any λ or t involve in the algorithm.
- L_1 arc length is defined as the sum of the changes of $\widehat{\beta}_j$ during the iteration.
- L_1 arc length plays a similar role as t.

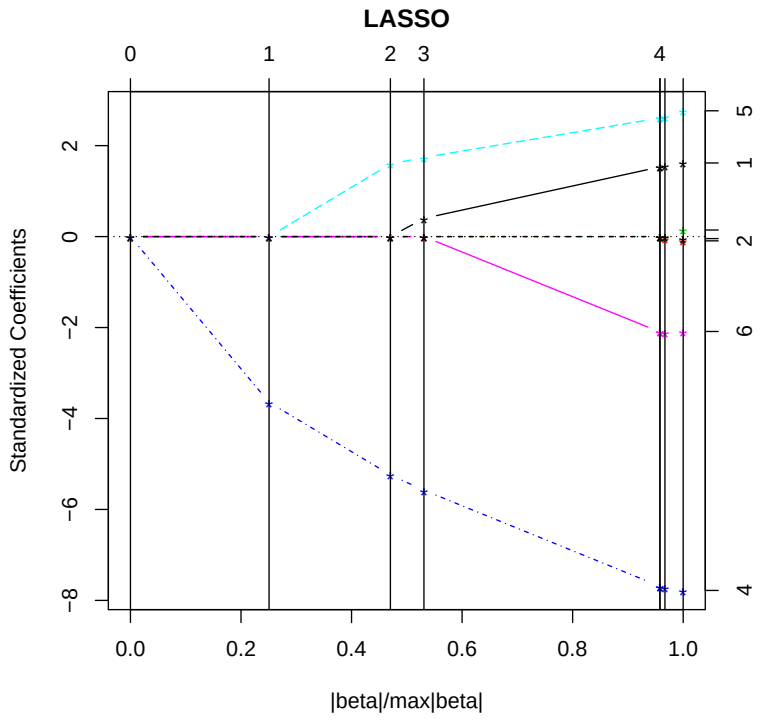
State Data

- You need to install R-packages “lars";
- The function *lars()* in the package performs Lasso regression.
- The first argument of *lars()* is the design matrix (without the intercept!);
- The second argument is the response variable;
- The parameter type=cc("lasso", "lar", "forward.stagewise", "stepwise") with default "lasso".
- Use “?lars" for more information;

```
library(lars)
data(state)
statedata <- data.frame(state.x77,row.names=state.abb)
lmod <- lars(as.matrix(statedata[,-4]),statedata$Life)
plot(lmod)## use ?plot.lars to check for details
coef(lmod)## Check the progress of coeffients
```

```
        Population        Income Illiteracy     Murder    HS.Grad        Frost          Area
[1,] 0.000000e+00  0.000000e+00 0.00000000  0.0000000 0.00000000  0.000000000  0.000000e+00
[2,] 0.000000e+00  0.000000e+00 0.00000000 -0.1410658 0.00000000  0.000000000  0.000000e+00
[3,] 0.000000e+00  0.000000e+00 0.00000000 -0.2027991 0.02821480  0.000000000  0.000000e+00
[4,] 1.280538e-05  0.000000e+00 0.00000000 -0.2158331 0.03078303  0.000000000  0.000000e+00
[5,] 4.899683e-05  0.000000e+00 0.00000000 -0.2975671 0.04609849 -0.005761311  0.000000e+00
[6,] 4.900136e-05 -5.216378e-08 0.00000000 -0.2975723 0.04610198 -0.005761624  0.000000e+00
[7,] 4.970871e-05 -8.191625e-06 0.00000000 -0.2982910 0.04669949 -0.005810432 -7.788778e-09
[8,] 5.180036e-05 -2.180424e-05 0.03382032 -0.3011232 0.04892948 -0.005735001 -7.383166e-08
```

State Data

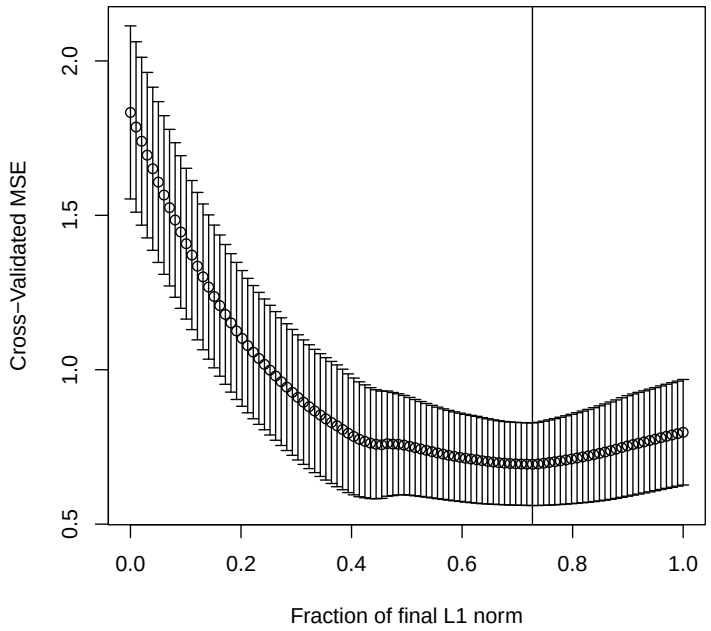


- X-axis for t is scaled by the maximum possible value for $\sum_{j=1}^{p} |\beta_j|$;
- For the smallest t, only predictor #4, the murder rate, is active;
- As t increases, predictor #5 enters;
- Predictor #1 and #6 enter later;
- The remaining three variables do not enter the model until t is very close to the least squares solution;
- As t increases, the size of the estimated coefficients also increase;

State Data

- The choice of t can be made using cross-validation;
- The default method for *lars()* uses a 10-fold cross-validation;
- The command *cv.lars()* can be used;
- Note: the optimal choice of t using this method is Random!

```
library(lars)
cvlmod <- cv.lars(as.matrix(statedata[,-4]),statedata$Life)
s<-cvlmod$index[which.min(cvlmod$cv)]
>0.4848485
abline(v=s)
```



State Data

- You can also use other criterions to choose the best model;
- *lars()* automatically computes Mallows C_p;
- You can compute AIC, BIC, GCV “by hands";

```
n <- dim(statedata)[1] #sample size
lmod_full <- lm(statedata$Life~as.matrix(statedata[,-4])) #full model
sigma_hat2 <- sum((lmod_full$residuals)^2)/(n-7-1)
RSS <- lmod$RSS #residual sum of squares for all models
df <- lmod$df #number of parameters in all models

AIC <- RSS/sigma_hat2+2*df
BIC <- RSS/sigma_hat2+log(n)*df
GCV <- RSS/n/(1-df/n)^2
Cp <- lmod$Cp
round(rbind(AIC,BIC,GCV, Cp),2)

         0     1     2     3     4     5     6     7
AIC 161.19 90.70 63.88 62.12 52.03 54.03 56.02 58.00
BIC 163.10 94.53 69.62 69.77 61.59 65.50 69.40 73.30
GCV   1.84  1.04  0.73  0.71  0.58  0.60  0.63  0.66
Cp  111.19 40.70 13.88 12.12  2.03  4.03  6.02  8.00
```

State Data

- After a lars/lasso model is fitted, you can use it for prediction;
- The R command for that is *predict()*
- Use “?predict.lars" to check more details;

```
library(lars)
obj <- predict(lmod,s=0.4848485,type="coef",mode="fraction")
>obj$coef
Population        Income    Illiteracy        Murder       HS.Grad
3.074070e-06  0.000000e+00  0.000000e+00 -2.059280e-01  2.883133e-02
Frost          Area
0.000000e+00  0.000000e+00
#Best model chosen by AIC, BIC, GCV
obj <- predict(lmod,s=5,type="coef",mode="step")
>obj$coef
   Population        Income    Illiteracy        Murder       HS.Grad
 4.899683e-05  0.000000e+00  0.000000e+00 -2.975671e-01  4.609849e-02
  Frost          Area
  -5.761311e-03  0.000000e+00
##Refit the selected model
coef(lm(Life.Exp ~ Population+Murder+HS.Grad+Frost, statedata))
 (Intercept)    Population        Murder       HS.Grad         Frost
 7.102713e+01  5.013998e-05 -3.001488e-01  4.658225e-02 -5.943290e-03
```

Refitted Lasso v.s. Lasso

- Lasso shrinks all β_j's toward 0, regardless of $\beta_j = 0$ or not;
- Hence, like the ridge regression estimator, $\widehat{\boldsymbol{\beta}}_{lasso}$ is also biased for $\boldsymbol{\beta}$;
- If you refit the selected model by LASSO, your estimated coefficients will be different!
- This is observed in our previous example;
- To see this more clearly, consider the case $\mathbf{X}^T\mathbf{X} = \mathbf{I}$, when $\widehat{\boldsymbol{\beta}}_{lasso}$ has a closed form

$$\widehat{\beta}_{lasso,j} = \begin{cases} \widehat{\beta}_{ols,j} - \lambda & \text{if } \widehat{\beta}_{lasso,j} > 0 \Leftrightarrow \widehat{\beta}_{ols,j} > \lambda \\ \widehat{\beta}_{ols,j} + \lambda & \text{if } \widehat{\beta}_{lasso,j} < 0 \Leftrightarrow \widehat{\beta}_{ols,j} < -\lambda \\ 0 & \text{if } \widehat{\beta}_{lasso,j} = 0 \Leftrightarrow |\widehat{\beta}_{ols,j}| \leq \lambda \end{cases}$$

- As long as $\lambda \neq 0$, $\hat{\beta}_{lasso,j}$ will be shrunk toward 0 even though it was eventually selected as non-zero coefficient;
- It is still debatable if the model selected by lasso should be refitted or not;

Loss+Penalty

- The regularized problem can be written as $\sum_{i=1}^{n} L(\boldsymbol{\beta}, y_i, x_i) + P_\lambda(\boldsymbol{\beta})$.
- The loss function $L(\boldsymbol{\beta}, y_i, x_i)$ measures goodness-of-fit of the model
  - Square loss: $(y_i - x_i^T \boldsymbol{\beta})^2$
  - Hinge loss: $(1 - y_i(x_i^T \boldsymbol{\beta}))_+$
  - Logistic loss: $\log(1 + e^{-y_i(x_i^T \boldsymbol{\beta})})$
- The penalty term $P_\lambda(\boldsymbol{\beta})$ controls the complexity of the model.
  - Ridge: $\lambda \sum_{j=1}^{p} \beta_j^2$
  - Lasso: $\lambda \sum_{j=1}^{p} |\beta_j|$
  - Elastic Net: $\lambda w \sum_{j=1}^{p} \beta_j^2 + \lambda(1-w) \sum_{j=1}^{p} |\beta_j|$ for some $w \in [0, 1]$
  - Smoothly Clipped Absolute Deviation (SCAD): $P_\lambda(\boldsymbol{\beta}) = \sum_{j=1}^{p} s_\lambda(|\beta_j|)$ with

$$s_\lambda(z) = \begin{cases} \lambda|z| & \text{if } |z| \le \lambda \\ -\left(\frac{|z|^2 - 2a\lambda|z| + \lambda^2}{2(a-1)}\right) & \text{if } \lambda < |z| \le a\lambda \text{ for some fixed } a > 2. \\ \frac{(a+1)\lambda^2}{2} & \text{if } |z| > a\lambda \end{cases}$$

  - Adaptive Lasso, Group Lasso, Fused Lasso. . . (You can pick a topic to present).

Coordinate Descent and "glmnet"

- So far, we have two algorithm to solve lasso problem.
  - QP: specify $t > 0$ such that $\sum_{j=1}^{p} |\beta_j| \leq t$.
  - Lars with lasso modification: do not specify t or λ.
- Given $\lambda \geq 0$, coordinate descent algorithm solves

$$\tfrac{1}{2}(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \textstyle\sum_{k=1}^{p} |\beta_k|.$$

- $\mathbf{X}_j$: the j-th column of $\mathbf{X}$.
- $\mathbf{X}_{-j}$: the columns of $\mathbf{X}$ except j-th column.
- $\boldsymbol{\beta}_{-j}$: the vector of $\boldsymbol{\beta}$ after removing β_j.
- The problem becomes

$$\tfrac{1}{2}(\boldsymbol{Y} - \mathbf{X}_{-j}\boldsymbol{\beta}_{-j} - \mathbf{X}_j\beta_j)^T(\boldsymbol{Y} - \mathbf{X}_{-j}\boldsymbol{\beta}_{-j} - \mathbf{X}_j\beta_j) + \lambda \textstyle\sum_{k=1}^{p} |\beta_k|.$$

- Taking derivative with respect to β_j for $j = 1, \ldots, p$, we have

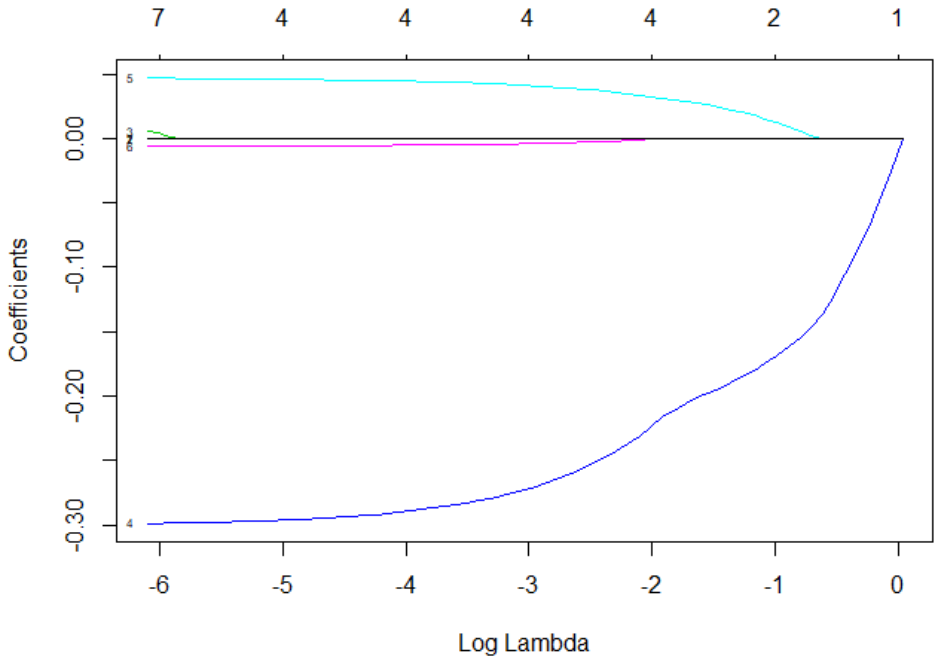
$$0 \in -\mathbf{X}_j^T(\boldsymbol{Y} - \mathbf{X}_{-j}\boldsymbol{\beta}_{-j} - \mathbf{X}_j\beta_j) + \lambda s_j, \text{ where } s_j = \begin{cases} 1 & \text{if } \beta_j > 0 \\ -1 & \text{if } \beta_j < 0 \\ [-1, 1] & \text{if } \beta_j = 0 \end{cases}.$$

Coordinate Descent and "glmnet"

- So we obtain $\beta_j = \begin{cases} \frac{\mathbf{X}_j^T(\boldsymbol{Y}-\mathbf{X}_{-j}\boldsymbol{\beta}_{-j})-\lambda}{\mathbf{X}_j^T\mathbf{X}_j} & \text{if } \mathbf{X}_j^T(\boldsymbol{Y}-\mathbf{X}_{-j}\boldsymbol{\beta}_{-j}) > \lambda \\ \frac{\mathbf{X}_j^T(\boldsymbol{Y}-\mathbf{X}_{-j}\boldsymbol{\beta}_{-j})+\lambda}{\mathbf{X}_j^T\mathbf{X}_j} & \text{if } \mathbf{X}_j^T(\boldsymbol{Y}-\mathbf{X}_{-j}\boldsymbol{\beta}_{-j}) < -\lambda \\ 0 & \text{if } |\mathbf{X}_j^T(\boldsymbol{Y}-\mathbf{X}_{-j}\boldsymbol{\beta}_{-j})| \leq \lambda \end{cases} \quad (*)$
- Similar, taking derivative with respect to β_0, we have
$$\beta_0 = \frac{1}{n}\sum_{i=1}^n (y_i - \sum_{k=1}^p \beta_k x_{ik}) \quad (**)$$
- Coordinate descent algorithm updates the value of each β_j iteratively:
  - Step 1: Initialize $(\beta_0, \beta_1, \ldots, \beta_p)$;
  - Step 2: for $j = 0, , \ldots, p$, treat $\boldsymbol{\beta}_{-j}$ as constant and update β_j according to $(*)$ and $(**)$;
  - Step 3: Repeat step 2 until convergence.
- For squared loss function, we have a closed form update formula.
- For other loss function, coordinate descent algorithm is usually combined with gradient descent or Newton's method.
- Coordinate descent algorithm is not as faster as Lars.
- But coordinate descent can be applied to a wider range of problem.
- R package "glmnet": based on coordinate descent (Newton's method).

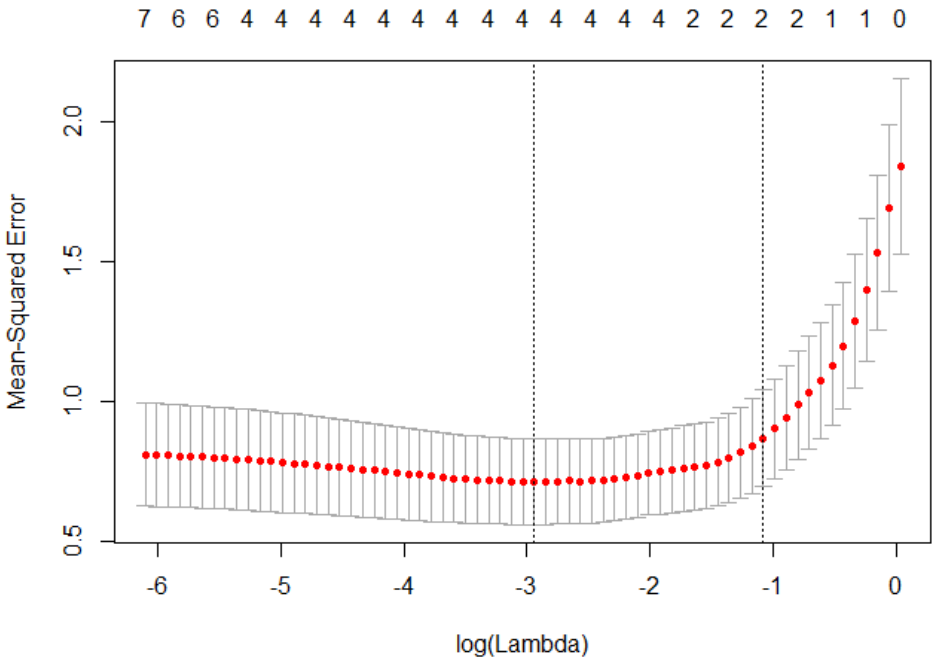
State Data

```
 library(lars)
library(glmnet)
data(state)
statedata=data.frame(state.x77,row.names=state.abb)

x=as.matrix(statedata[,-4])
y=statedata$Life

fit=glmnet(x,y)
plot(fit,label = T, xvar="lambda")
```



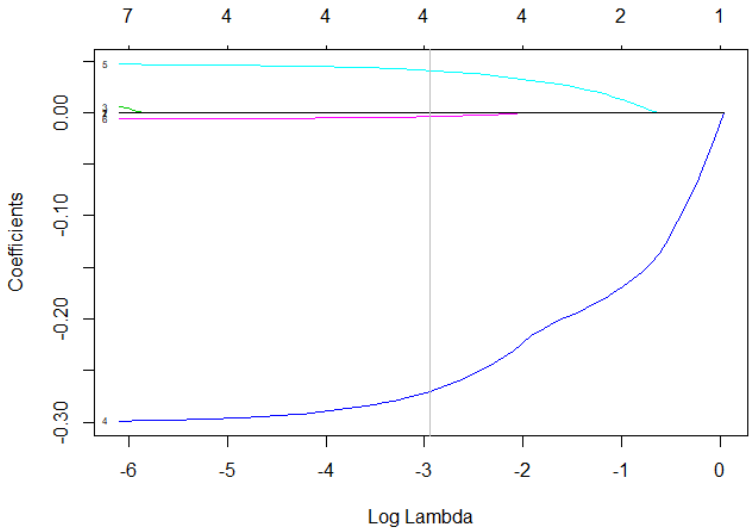
glmnet: state data

```
cv_fit=cv.glmnet(x,y,nfolds = length(y),grouped = FALSE) #

plot(cv_fit)

lambda_opt=cv_fit$lambda.min
> lambda_opt
[1] 0.05286021
```



State Data

```
plot(fit,label = T, xvar="lambda")
abline(v=log(lambda_opt),col="gray")
final_fit=glmnet(x,y,lambda=lambda_opt)
cf=coef(final_fit)
> cf
8 x 1 sparse Matrix of class "dgCMatrix"
                      s0
(Intercept)  7.093953e+01
Population   3.682775e-05
Income       .
Illiteracy   .
Murder      -2.700762e-01
HS.Grad      4.094966e-02
Frost       -3.823926e-03
Area         .
```



State Data

```
index=which(cf[,1]!=0)
fml="Life.Exp~"
for (i in index){
  if (i>1){
    fml=paste(fml,"+",rownames(cf)[i],sep="")
  }
}
fml=formula(fml)
> fml
Life.Exp ~ +Population + Murder + HS.Grad + Frost

re_fit=lm(fml,data=statedata)
> summary(re_fit)

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  7.103e+01  9.529e-01  74.542  < 2e-16 ***
Population   5.014e-05  2.512e-05   1.996  0.05201 .
Murder      -3.001e-01  3.661e-02  -8.199 1.77e-10 ***
HS.Grad      4.658e-02  1.483e-02   3.142  0.00297 **
Frost       -5.943e-03  2.421e-03  -2.455  0.01802 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Adaptive Lasso

- Adaptive lasso uses weights to penalize different coefficients:
$$(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta})^T(\boldsymbol{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \sum_{j=1}^{p} |\widehat{\beta}_j|^{-\gamma} |\beta_j|$$
for some $\lambda \geq 0, \gamma > 0$;
- $\widehat{\beta}_1, \ldots, \widehat{\beta}_p$ are pilot estimates: ridge or OLS
- $\gamma = 0.5, 1, 2$: adjustment of the weights vector
- Adaptively penalize more those coefficients with lower pilot estimates.

State Data

```
rm(list=ls())
library(glmnet)
set.seed(123)
data(state)
statedata=data.frame(state.x77,row.names=state.abb)
x=as.matrix(statedata[,-4])
y=statedata$Life

cv.ridge=cv.glmnet(x, y, alpha=0)
cf.pilot=coef(cv.ridge, s=cv.ridge$lambda.min)
cf.pilot=cf.pilot[-1] #remove intercept
gamma=1
w=1/(abs(cf.pilot))^gamma #weights

cv.lasso=cv.glmnet(x, y, alpha=1, penalty.factor=w, lambda = exp(seq(-5,0, length=100)))
plot(cv.lasso)
plot(cv.lasso$glmnet.fit, xvar="lambda", label=TRUE)
abline(v = log(cv.lasso$lambda.min))
coef(cv.lasso, s=cv.lasso$lambda.min)

(Intercept)  7.116439e+01
Population   3.161932e-05
Income        .
Illiteracy  -5.046627e-02
Murder      -2.910325e-01
HS.Grad      4.642217e-02
Frost       -6.501633e-03
Area          .
```

Dimension Reduction

- Consider the multiple regression model

$$y_i = \beta_0 + x_1\beta_1 + \cdots + x_p\beta_p + \varepsilon_i;$$

- When we have many potential predictors, what should we do?
- Model selection: select the best subset of $x_1, x_2, \ldots, x_p$ as your “best" model;
- Model selection methods: subset selection, stepwise model selection, Lasso, etc.
- Is there any other way? Dimension reduction;
  - Many predictors $x_1, x_2, \ldots, x_p$ contain similar information;
  - Can we create a few new predictors that can absorb most essential information in $x_1, x_2, \ldots, x_p$?
  - At the same time, these new predictors contain completely different information?
- Above is the motivation of the so-called “principal components".

Create New Predictors

- The form of the new predictors takes the following form

$$z_j = u_{1j}x_1 + u_{2j}x_2 + \cdots + u_{pj}x_p$$

  $j = 1, \ldots, p$;
- $z_1, z_2, \ldots, z_p$ are our new “predictors";
- How do we choose all the weights u_{ij}'s?
- RULE 1: z_1 absorb as much of information in x_j's as possible;
  - In statistics, variability of a predictor measures its information;
  - Hence, the $\boldsymbol{u}_1 = (u_{11}, u_{12}, \ldots, u_{1p})^T$ can be chosen such that

$$Var(\boldsymbol{u}_1^T \boldsymbol{x}) \text{ is maximized subject to } \quad \boldsymbol{u}_1^T \boldsymbol{u}_1 = 1$$

    where $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)^T$.
- RULE 2: $z_2 = \boldsymbol{u}_2^T \boldsymbol{x}$ should contain different information from $z_1 = \boldsymbol{u}_1^T \boldsymbol{x}$;
  - We need $\boldsymbol{u}_1^T \boldsymbol{u}_2 = 0$, where $\boldsymbol{u}_2^T \boldsymbol{u}_2 = 1$
  - Later we will show that, the solution of PCA satisfies $Cov(z_1, z_2) = 0$;

Principal Components – Algorithm

- The algorithm to find all z_j's are as follows
  - Find the $\boldsymbol{u}_1$ such that $var(\boldsymbol{u}_1^T \boldsymbol{x})$ is maximized subject to $\boldsymbol{u}_1^T \boldsymbol{u}_1 = 1$.
    - z_1-the first principal component.
  - Find $\boldsymbol{u}_2$ such that $var(\boldsymbol{u}_2^T \boldsymbol{x})$ is maximized subject to $\boldsymbol{u}_1^T \boldsymbol{u}_2 = 0$ and $\boldsymbol{u}_1^T \boldsymbol{u}_1 = 1$.
    - z_2-the second principal component.
  - Find $\boldsymbol{u}_3$ such that $var(\boldsymbol{u}_3^T \boldsymbol{x})$ is maximized subject to $\boldsymbol{u}_3^T \boldsymbol{u}_1 = 0$, $\boldsymbol{u}_3^T \boldsymbol{u}_2 = 0$, and $\boldsymbol{u}_3^T \boldsymbol{u}_3 = 1$.
    - z_3-the third principal component.
  - Keep finding directions of greatest variation orthogonal to those directions we have already found.

Principal Components – Solution

- The solution to the previous algorithm is
- Let $\Sigma = Cov(\boldsymbol{x})$: the $p \times p$ covariance matrix of random vector $\boldsymbol{x} = (x_1, x_2, \ldots, x_p)^T$;
  - Find the largest eigen-value of Σ, say λ_1, and the corresponding eigen-vector, $\boldsymbol{u}_1$;
    - The first principal component: $z_1 = \boldsymbol{u}_1^T \boldsymbol{x}$ and $Var(z_1) = \lambda_1$;
  - Find the second largest eigen-value of Σ, say λ_2, and the corresponding eigen-vector, $\boldsymbol{u}_2$;
    - The second principal component: $z_2 = \boldsymbol{u}_2^T \boldsymbol{x}$ and $Var(z_2) = \lambda_2$;
  - $\vdots$
  - Find the pth largest eigen-value of Σ, say λ_p, and the corresponding eigen-vector, $\boldsymbol{u}_p$;
    - The pth principal component: $z_p = \boldsymbol{u}_p^T \boldsymbol{x}$ and $Var(z_p) = \lambda_p$;
- In addition, $Cov(z_i, z_j) = 0$ if $i \neq j$;
- $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$;

Principal Components – Proof

Proof.

Let's look at the first principal component. The goal is to maximize

$$Var(\boldsymbol{u}_1^T \boldsymbol{x}) = \boldsymbol{u}_1^T Var(\boldsymbol{x}) \boldsymbol{u}_1 = \boldsymbol{u}_1^T \Sigma \boldsymbol{u}_1 \text{ s.t. } \boldsymbol{u}_1^T \boldsymbol{u}_1 = 1;$$

Then using the lagrange multiplier, it is equivalent to maximize

$$\ell(\boldsymbol{u}_1) = \boldsymbol{u}_1^T \Sigma \boldsymbol{u}_1 + \gamma(\boldsymbol{u}_1^T \boldsymbol{u}_1 - 1).$$

Take the first partial derivative (in matrix form)

$$\nabla \ell(\boldsymbol{u}_1) = 2\Sigma \boldsymbol{u}_1 + 2\gamma \boldsymbol{u}_1 = 0$$

Solve for $\boldsymbol{u}_1$, we must have $\Sigma \boldsymbol{u}_1 = -\gamma \boldsymbol{u}_1$, which means $\boldsymbol{u}_1$ must be an eigen-vector of Σ. Since as an eigen-vector

$$\boldsymbol{u}_1^T \Sigma \boldsymbol{u}_1 = \lambda_1$$

with λ_1 being an eigen-value of Σ. To maximize $\boldsymbol{u}_1^T \Sigma \boldsymbol{u}_1$, λ_1 must be the largest eigen-value of Σ. $\square$

Principal Components – Proof (not required)

Proof.

Now let's look at the second principal component. The goal is to maximize

$$Var(\boldsymbol{u}_2^T \boldsymbol{x}) = \boldsymbol{u}_2^T \Sigma \boldsymbol{u}_2 \text{ s.t. } \boldsymbol{u}_1^T \boldsymbol{u}_2 = 0, \boldsymbol{u}_2^T \boldsymbol{u}_2 = 1;$$

Then using the lagrange multiplier, it is equivalent to maximize

$$\ell(\boldsymbol{u}_2) = \boldsymbol{u}_2^T \Sigma \boldsymbol{u}_2 + \gamma_1(\boldsymbol{u}_2^T \boldsymbol{u}_2 - 1) + \gamma_2 \boldsymbol{u}_1^T \boldsymbol{u}_2.$$

Take the first partial derivative (in matrix form)

$$\nabla \ell(\boldsymbol{u}_2) = 2\Sigma \boldsymbol{u}_2 + 2\gamma_1 \boldsymbol{u}_2 + \gamma_2 \boldsymbol{u}_1 = 0$$

Multiply $\boldsymbol{u}_1^T$ on both sides of above equation we have

$$2 \underbrace{\boldsymbol{u}_1^T \Sigma \boldsymbol{u}_2}_{\lambda_1 \boldsymbol{u}_1^T \boldsymbol{u}_2 = 0} + 2\gamma_1 \underbrace{\boldsymbol{u}_1^T \boldsymbol{u}_2}_{=0} + \gamma_2 \boldsymbol{u}_1^T \boldsymbol{u}_1 = 0$$

Hence $\gamma_2 = 0$. □

Principal Components – Proof (not required)

Proof.

- Then we must have
$$2\Sigma \boldsymbol{u}_2 + 2\gamma_1 \boldsymbol{u}_2 = 0,$$
which means $\boldsymbol{u}_2$ is also an eigen-vector of Σ. Since as an eigen-vector
$$\boldsymbol{u}_2^T \Sigma \boldsymbol{u}_2 = \lambda_2$$
where λ_2 is another eigen-value of Σ.
- We need to ensure that $\boldsymbol{u}_2^T \boldsymbol{u}_1 = 0$;
- To maximize $\boldsymbol{u}_2^T \Sigma \boldsymbol{u}_2$, λ_2 must be the 2nd largest eigen-value of Σ.
- In addition,
$$Cov(z_1, z_2) = Cov(\boldsymbol{u}_1^T \boldsymbol{x}, \boldsymbol{u}_2^T \boldsymbol{x}) = \boldsymbol{u}_1^T \Sigma \boldsymbol{u}_2 = \lambda_1 \boldsymbol{u}_1^T \boldsymbol{u}_2 = 0$$
hence z_1, z_2 are uncorrelated!
- The same argument can be applied for $z_3, z_4, \ldots, z_p$, proof completed.

□

Principal Components – Implementation

- To compute all principal components, we need to know the covariance matrix $\Sigma = Cov(\boldsymbol{x})$;
- However, Σ is unknown; we can use the sample covariance matrix instead

$$\mathbf{S} = \frac{1}{n-1}\sum_{i=1}^{n}\{(\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^T\}$$

- R function *prcomp()* can be used to compute principal components;
- Use “?prcomp()" to find out more information of this function;

```
library(faraway)
library(MASS)
data(fat,package="faraway")
cfat <- fat[,9:18]
prfat <- prcomp(cfat)
##the rotation matrix-columns of u_i's
prfat$rot
##The sqrt(lambda_i)'s
prfat$sdev
```

How Many Principal Components?

- How many PCs do we need?
- The proportion of total variance explained by the jth PC is

$$\text{Proportion of Variance}_j = \frac{\lambda_j}{\lambda_1 + \lambda_2 + \cdots + \lambda_p}$$

- The cumulative proportion of total variance explained by the first j PCs is

$$\text{Cumulative Proportion}_j = \frac{\lambda_1 + \cdots + \lambda_j}{\lambda_1 + \lambda_2 + \cdots + \lambda_p}$$

- Both can be used to help decide how many PCs needed. (but neither of them is NOT a golden criterion.)
- The usual goal is to choose the number beyond which all the PC standard deviations are relatively small;

How Many Principal Components?

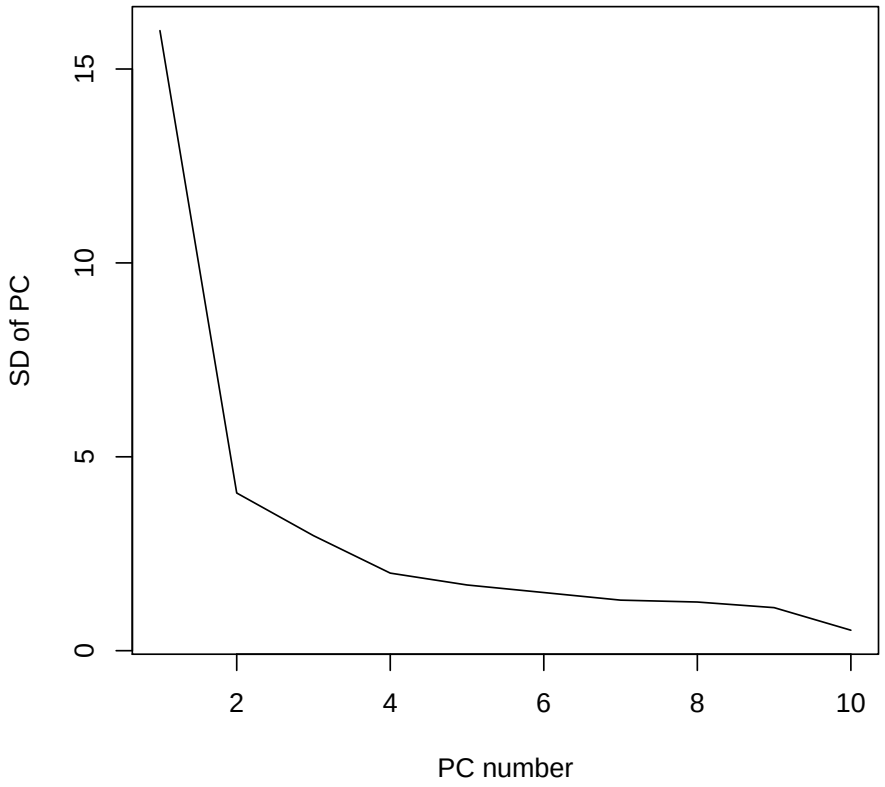
```
prfat <- prcomp(cfat)
>summary(prfat)
Importance of components:
          PC1     PC2     PC3     PC4     PC5
Standard deviation     15.990 4.06584 2.96596 2.00044 1.69408
Proportion of Variance  0.867 0.05605 0.02983 0.01357 0.00973
Cumulative Proportion   0.867 0.92304 0.95287 0.96644 0.97617
         PC6     PC7     PC8     PC9    PC10
Standard deviation     1.49881 1.30322 1.25478 1.10955 0.52737
Proportion of Variance 0.00762 0.00576 0.00534 0.00417 0.00094
Cumulative Proportion  0.98378 0.98954 0.99488 0.99906 1.00000
```

- The summary gives the standard deviations of the principal components $z_j = \boldsymbol{u}_j^T \boldsymbol{x}$ for $j = 1, \ldots, 10$. (i.e. $\sqrt{\lambda_j}$).
- The first PC explains 86.7% of the variation in the data;
- The first three PCs accounts for 95.3% of the variation in the data;
- In this case, using the First or the first two PCs should be enough;

How Many Principal Components?

- A good way to determine this number is to make a scree plot: an index plot of the standard deviations of PCs (i.e. $\sqrt{\lambda_j}$'s);
- A visual examination suggests that maybe 4 PCs will be enough;

```
prfat <- prcomp(cfat)
plot(prfat$sdev,type="l",ylab="SD of PC", xlab="PC number")
```



Principal Components – Standardization?

- Take a look at the 1st PC of the “fat" data;

```
>round(prfat$rot[,1],2)
neck chest abdom  hip thigh  knee  ankle  biceps forearm  wrist
0.12 0.50  0.66  0.42  0.28  0.12  0.06    0.15    0.07   0.04
```

- The chest, abdomen, hip and thigh measures dominate the first principal component.
- Why would this happen? Take a look at the variance of each variable

```
round(apply(cfat,2,var),2)
neck   chest   abdom   hip   thigh   knee ankle biceps forearm wrist
5.91   71.07   116.27  51.32  27.56  5.82 2.87  9.13    4.08    0.87
```

- The variance of chest, abdomen, hip and thigh dominate others;
- As a result, they also dominate the 1st PC;
- Recall that the principal components aims at maximizing $Var(z_j)$ where $z_j = u_{1j}x_1 + \cdots + u_{pj}x_p$, hence x_j's with large variances will get higher weights (u_j's, intuitively.)
- A direct consequence is, the unit of x_j's matters! (for example “price" in cents has 100^2 times greater variance than the same “price" meaured in “dollors". NOT GOOD!)

Principal Components – Standardization?

- One solution to “uneven variance" issue: standardize all variables;
- The original principal components can be obtained from the eigen-decomposition of the sample covariance matrix

$$\mathbf{S} = \frac{1}{n-1}\sum_{i=1}^{n}\{(\boldsymbol{x}_i - \overline{\boldsymbol{x}})(\boldsymbol{x}_i - \overline{\boldsymbol{x}})^T\}$$

- Standardization: $x_j^* = x_j / sd(x_j)$;
- If we choose to standardize all variables first, the new principal components can be obtained from the eigen-decomposition of the sample correlation matrix

$$\mathbf{R} = \frac{1}{n-1}\sum_{i=1}^{n}\{(\boldsymbol{x}_i^* - \overline{\boldsymbol{x}^*})(\boldsymbol{x}_i^* - \overline{\boldsymbol{x}^*})^T\}$$

- Standardization in *prcomp()* function: set “scale=TRUE";

```
prfatc <- prcomp(cfat, scale=TRUE)
round(prfatc$rot[,1],2)
neck  chest  abdom  hip   thigh  knee ankle biceps forearm  wrist
0.33  0.34   0.33   0.35   0.33  0.33 0.25   0.32   0.27    0.30
```

- Weights in the 1st PC becomes quite even.

Principal Components – Interpretation

- Sometime, the principal components z_j's may have nice interpretation;
- Unfortunately, this does not happen very often;
  - x_j's measures very different things; e.g. what does it mean by “age+weight"? hard to integrate;
  - But even without a nice interpretation, PCs can still be useful;
- If you find a PC that has a very nice interpretation, it is valuable;

```
prfatc <- prcomp(cfat, scale=TRUE)
round(prfatc$rot[,1],2) ##Fist principal component
neck  chest  abdom  hip   thigh  knee ankle biceps forearm  wrist
0.33  0.34   0.33   0.35   0.33  0.33 0.25   0.32   0.27    0.30
round(prfatc$rot[,2],2) ##Second principal component
neck  chest  abdom  hip   thigh  knee  ankle biceps forearm wrist
0.00  -0.27  -0.40  -0.25 -0.19  0.02  0.62  0.02    0.36    0.38
```

- First PC: interpret this as overall size since it is roughly proportional to a mean across all these (standardized) variables.
- Second PC: roughly a contrast between the body center measures of chest, abdomen, hip and thigh circumferences against the extremities of forearm, wrist and ankle.

Some Comments

- As a dimension reduction tool, the principal components analysis is useful;
- There are some unsatisfactory aspects of PCA:
  - The interpretation of PCs is often subjective;
  - Choice of number of PCs can be subjective;
  - Sensitive to outliers;
  - Standardization v.s. original data?
    - If x_j's with roughly the same variance, don't do standardization;
    - If some components are domination the variance, standardize;
  - All x_j's must be numerical! But in practice, data can be categorical or mixture of numerical and categorical data!

Principal Components Regression

- The definition of principal components does not depend on the response variable y;
- The idea is to find a way to re-organize the “information" in $x_1, x_2, \ldots, x_p$ by taking linear combinations;
- There are at most p PCs: $z_1, z_2, \ldots, z_p$;
- $Var(z_j) = \lambda_j$ with λ_j being the jth largest eigen-value of the covariance matrix $\Sigma = Cov(\boldsymbol{x})$;
- All PCs are uncorrelated: $Cov(z_i, z_j) = 0$ for any $i \neq j$;
- Original regression model

$$y_i = \beta_0 + x_1\beta_1 + \cdots + x_p\beta_p + \varepsilon_i;$$

- Principal components regression model (PCR)

$$y_i = \beta_0 + z_1\beta_1 + \cdots + z_K\beta_K + \varepsilon_i;$$

  where $z_1, \ldots, z_K$ are first K PCs. Typically, $K < p$;
- Benefits: we can use a few number of predictors; (in a sense, similar to model selection.)

Body Fat Data

- Why would abdomen circumference have a positive effect while hip circumference has negative effect?

```
library(faraway)
cfat <- fat[,9:18]
lmoda <- lm(fat$brozek ~ ., data=cfat)
sumary(lmoda)
Estimate Std. Error t value  Pr(>|t|)
(Intercept)  7.2287487  6.2143092  1.1632 0.2458816
neck        -0.5819470  0.2085800 -2.7900 0.0056916
chest       -0.0908468  0.0854300 -1.0634 0.2886622
abdom        0.9602291  0.0715821 13.4144 < 2.2e-16
hip         -0.3913546  0.1126862 -3.4730 0.0006101
thigh        0.1337081  0.1249222  1.0703 0.2855412
knee        -0.0940552  0.2123939 -0.4428 0.6582831
ankle        0.0042223  0.2031754  0.0208 0.9834370
biceps       0.1111963  0.1591179  0.6988 0.4853321
forearm      0.3445364  0.1855113  1.8572 0.0644989
wrist       -1.3534719  0.4714098 -2.8711 0.0044542
n = 252, p = 11, Residual SE = 4.07132, R-Squared = 0.74
> round(vif(lmoda),2) ##Find VIF
neck  chest  abdom   hip   thigh   knee   ankle  biceps forearm   wrist
3.89  7.85   9.02    9.87  6.51    3.97   1.80    3.50    2.13    2.93
```

Body Fat Data

- Obviously, you have observed serious Multi-collinearity;
- Previously, you can remove some predictors to reduce Multi-collinearity;
- Principal component regression: effective in dealing with Multi-collinearity;

```
library(faraway)
library(MASS)
data(fat,package="faraway")
cfat <- fat[,9:18]
##PCA with standardized data
prfatc <- prcomp(cfat, scale=TRUE)
##PC regression with first two PCs
lmodpcr <- lm(fat$brozek ~ prfatc$x[,1:2])
sumary(lmodpcr)
Estimate Std. Error t value  Pr(>|t|)
(Intercept)        18.93849    0.32913 57.5416 < 2.2e-16
prfatc$x[, 1:2]PC1  1.84198    0.12446 14.8003 < 2.2e-16
prfatc$x[, 1:2]PC2 -3.55053    0.38661 -9.1837 < 2.2e-16
n = 252, p = 3, Residual SE = 5.22473, R-Squared = 0.55
```

Body Fat Data

- You have probably noticed, using only first two PCs, the R^2 reduced from 74% to 55%;
- Maybe you are not happy with this “loss", you can try regression with five PCs; $R^2 = 0.62$;
- However, PC3 and PC4 are insignificant;
- Question: can we do model selection on PCs? No, last few PCs have variance close to 0, behave like constants (or intercept);

```
library(faraway)
library(MASS)
##PC regression with first three PCs
lmodpcr <- lm(fat$brozek ~ prfatc$x[,1:5])
> sumary(lmodpcr)
Estimate Std. Error t value  Pr(>|t|)
(Intercept)        18.93849    0.30341 62.4188 < 2.2e-16
prfatc$x[, 1:5]PC1  1.84198    0.11473 16.0548 < 2.2e-16
prfatc$x[, 1:5]PC2 -3.55053    0.35640 -9.9621 < 2.2e-16
prfatc$x[, 1:5]PC3  0.25669    0.37116  0.6916    0.4898
prfatc$x[, 1:5]PC4  0.54094    0.43360  1.2476    0.2134
prfatc$x[, 1:5]PC5  3.72632    0.55571  6.7056 1.365e-10
n = 252, p = 6, Residual SE = 4.81649, R-Squared = 0.62
```

How Many Principal Components for Regression?

- How many PCs do we need for regression?
- We have dealt with this issue before, we can look at a “scree" plot;
- But when the response variable y is involved, this becomes a different questions;
- How many PCs are necessary depends on how well these PCs do in terms of explaining variations in Y;
- In other words, it depends on how well selected PCs can predict Y;
- However, if you only look at R^2, the more PCs, the better (NOT good);
- How can we honestly evaluate prediction performance? Cross-validation!
- If successful, we can sometimes make better predictions with a small number of PCs than a much larger number of predictors $\boldsymbol{x}$;

Meatspec Data

- Samples of finely chopped pure meat and 215 samples were measured;
- For each sample, the fat content was measured along with a 100-channel spectrum of absorbances;
- We would like to build a model to predict the fat content of new samples using the 100 absorbances;
- We have 100 predictors (x_i's) and $n = 215$ observations;
- Partition the data into two parts:
  - a training sample: first 172 observations to fit the model;
  - a testing sample: the remaining 43 observations.

```
data(meatspec, package="faraway")
trainmeat <- meatspec[1:172,]
testmeat <- meatspec[173:215,]
modlm <- lm(fat ~ ., trainmeat)
> summary(modlm)$r.squared
[1] 0.99702
```

Meatspec Data

- The perfect R^2 on the training data set means nothing!
- We have 172 data points and 100 predictors;
- Over-fitting is most likely;
- Let's calculate the Root-MSE: $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$;
- RMSE is a lot worse on Test data than the training data: an indication of over-fitting

```
##RMSE on the training data
rmse <- function(x,y) sqrt(mean((x-y)^2))
rmse(fitted(modlm), trainmeat$fat)
[1] 0.6903167
##RMSE on the test data
rmse(predict(modlm,testmeat), testmeat$fat)
[1]  3.814
```

Meatspec Data

- Let's try model selection first;
- We use stepwise selection with AIC, 32 predictors were selected;
- Using the selected model, the training MSE becomes worse (1.154551);
- But the MSE on the test data becomes better (2.424905);
- It indicates: model selection help us reduced overfitting;

```
library(Rcmdr)
modlm <- lm(fat ~ ., trainmeat)
obj<-stepwise(modlm,direction="forward/backward",criterion="AIC")
##RMSE on the training data
rmse <- function(x,y) sqrt(mean((x-y)^2))
rmse(fitted(obj), trainmeat$fat)
[1]  1.154551
##RMSE on the test data
rmse(predict(obj,testmeat), testmeat$fat)
[1]  2.424905
```

Meatspec Data

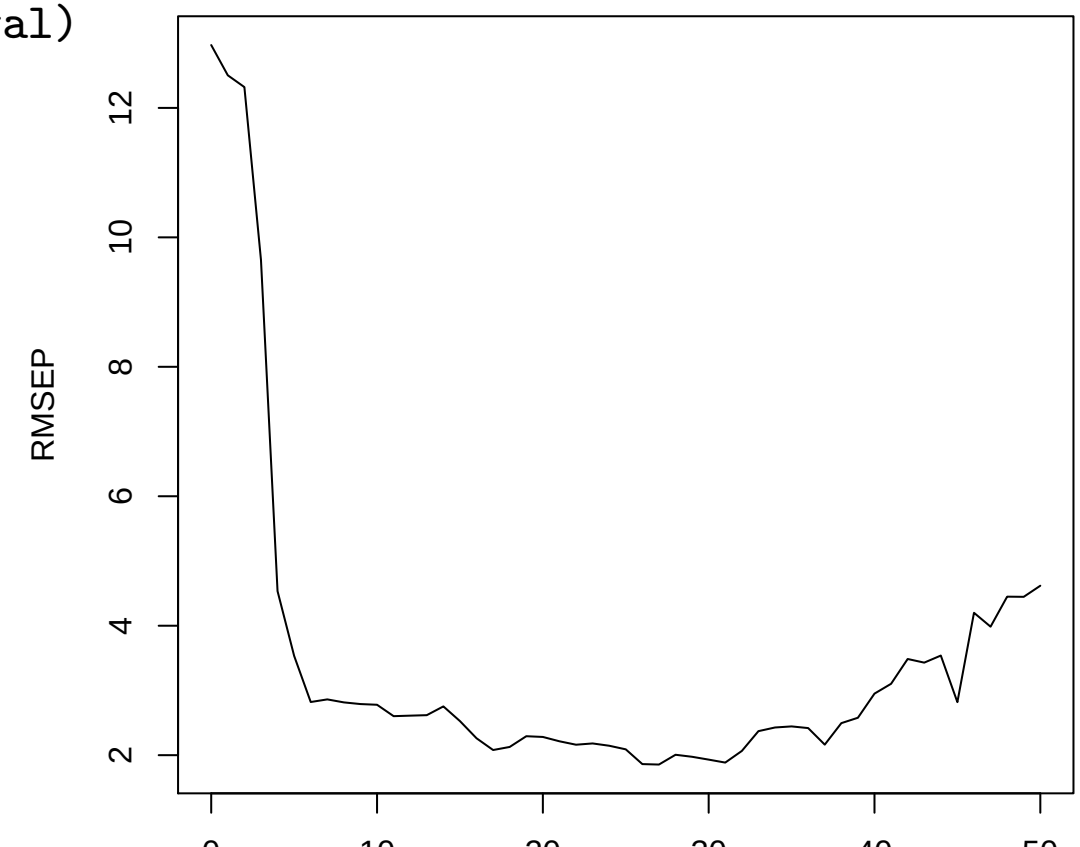
- Now let's try to use PCR on "meatspec" data;
- First PC accounts for about ten times more variation than the second.
- Most of the variation in the predictors can be explained with just a few dimensions.
- PCR is suitable for this data set;

```
##compute the PCA on the training sample predictors
meatpca <- prcomp(trainmeat[,-101])
##examine the standard deviations of the principal components
round(meatpca$sdev,3)
[1] 5.055 0.511 0.282 0.168 0.038 0.025 0.014 0.011 0.005 0.003 0.002 0.002
[13] 0.001 0.001 0.001 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[25] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[37] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[49] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[61] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[73] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[85] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
[97] 0.000 0.000 0.000 0.000
```

Meatspec Data

- Function *pcr()* in Package “pls": principal component regression;
- Use “?pcr" to find more info on *pcr()*;

```
require(pls)
#PCR with at most 50 PCs
pcrmod <- pcr(fat ~ ., data=trainmeat, ncomp=50)
##Compute MSE on test data using different number of PCs
pcrmse <- RMSEP(pcrmod, newdata=testmeat)
plot(pcrmse,main="")
> which.min(pcrmse$val)
[1] 28
> pcrmse$val[28]
[1] 1.8549
```



Meatspec Data

- The MSE on test data were minimized with 28 PCs;
- MSE on test data is better than model selection; (But this is not a fair comparison, why?)
- The *pcr()* provides Cross-validation it self;
- The downside of Cross-validation: the results changes from time to time since the partition of your data is random!

```
pcrmod <- pcr(fat ~ ., data=trainmeat, validation="CV", ncomp=50)
pcrCV <- RMSEP(pcrmod, estimate="CV")
plot(pcrCV,main="")
which.min(pcrCV$val)
[1] 22
#Prediction of PCR with 22 PCs
ypred <- predict(pcrmod, testmeat, ncomp=22)
rmse(ypred, testmeat$fat)
[1] 2.161623
```